

Modely kontextu (Context models)

Zdeněk VAŠÍČEK

Fakulta informačních technologií, Vysoké učení technické v Brně

Brno, Czech Republic

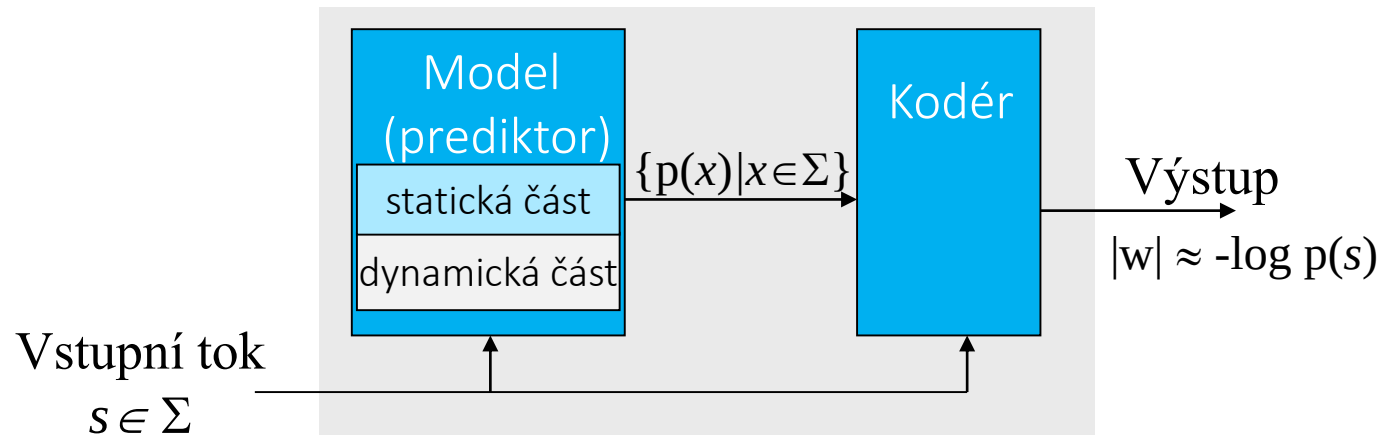
vasicek@fit.vutbr.cz

Obsah

- **Modely kontextu a obecná statistická metoda komprese dat**
 - Definice, princip, parametry, klasifikace modelů
- **Modely kontextu s pevným řádem**
 - Adaptivní model pracující po bitech
 - Adaptivní model pracující po bytech
- **Modely kontextu s proměnným řádem**
 - Model pracující po bytech
 - PPM (Partial Prediction Matching)
 - Modely pracující po bitech:
 - DMC (Dynamic Markov Coding)
 - CTW (Context Tree Weighting)
- **Mixování kontextů**
- **PAQ**

Model

- Techniky entropického kódování jako je Huffmanovo či aritmetické kódování jsou založeny na znalosti pravděpodobnostního rozložení bez kterého se neobejdou.



- Obecná statistická kompresní metoda je složena z **modelovacího stupně následovaného kódovacím stupněm**. Model přiděluje vstupním symbolům **pravděpodobnosti** a kódovací stupeň **pak kóduje** symboly na základě jejich pravděpodobností.
 - Model může být statický, nebo dynamický (adaptivní).
 - Máme-li k dispozici vhodný model dat, kódování je vyřešený problém (model lze použít k predikci dalšího symbolu na vstupu).

Model

- Modelem rozumíme odhad rozložení pravděpodobnosti výskytu dat ve vstupním toku.
 - Model může pracovat nad: bity, byty, slovy, ...
- Míra efektivity kodéru záleží na kvalitě pravděpodobnostního modelu.
 - Čím více se model vzdaluje skutečnosti, tím méně je kódér efektivní. Snahou všech pokročilých metod komprese je automaticky vytvořit co nejuvhodnější model.
 - Jak však teoreticky dokázal Kolmogorov, *optimální univerzální model neexistuje*.
- Většina modelů je založena na jednom ze dvou principů
 - **Statistický model:** Model přiřadí pravděpodobnosti symbolům textu podle jejich četnosti (počtu) výskytu tak, že často se vyskytující symboly dostanou krátké kódy. Statický model používá konstantní pravděpodobnosti, zatímco dynamický model upravuje pravděpodobnosti v průběhu komprese.
 - **Kontextově orientovaný model:** Při přidělování pravděpodobností bere model v úvahu kontext symbolu. Protože dekodér nemá k dispozici budoucí text, musí kódér i dekodér omezit kontext na minulý text, tj. na symboly, které již vstoupily a byly zpracovány. V praxi je kontext symbolu tvořen N symboly, které jej předcházejí. Proto říkáme, že kompresní metoda založená na kontextu používá kontextu symbolu k jeho **predikci** (tj. přidělí mu pravděpodobnost).

Řád modelu

- Řádem modelu rozumíme *počet symbolů bezprostředně předcházejících právě zpracovávaný symbol* použitých k určení pravděpodobnostního rozložení
- Model nultého řádu
 - nejjednodušší model, kdy uvažujeme pouze pravděpodobnost výskytu izolovaných symbolů $P(s)$, kde $s \in \Sigma$ jsou jednotlivé symboly vstupní sekvence
 - entropie modelu nultého řádu je definována jako

$$\mathcal{H}_0(T) = - \sum_{s \in \Sigma} P(s) \log_2 P(s)$$

- Příkladem metody pracující na bázi modelu nultého řádu je Huffmanův kód
- Modely vyššího řádu
 - v přirozeném jazyce je velká korelace mezi sousedními symboly (viz následující slide), vyšší řád proto bude dosahovat lepšího kompresního poměru
 - můžeme lépe predikovat následující symbol; čím vyšší úspěšnost predikce, tím více klesá entropie dat a tím pádem můžeme dosáhnout lepšího kompresního poměru

Model vyššího řádu

■ Model prvního řádu

- vede na podmíněnou pravděpodobnost.
- entropie modelu prvního řádu odpovídá podmíněné entropii $H(S|C)$ uvažujeme-li abecedy S a C

$$\mathcal{H}(S|C) = - \sum_{s \in S} \sum_{c \in C} P(s|c)P(c) \log_2 P(s|c)$$

- Podmíněnou entropii lze chápat jako množství nejistoty, které máme o S , známe-li náhodnou veličinu C .
- Obecně platí, že jakákoliv dodatečná informace o C redukuje entropii S , tj. platí

$$\mathcal{H}(S|C) \leq \mathcal{H}(S)$$

■ Příklad: anglicky psaný text

- v přirozeném jazyce $P('H') = 5\%$, znak $P('T') = 9\%$
- pokud je však v textu znak T , s více než 30% pravděpodobností bude následující znak H (digram TH je velice běžný) $P('H'|'T')=30\%$. Naopak, pokud je v textu znak X , pravděpodobnost, že bude následovat H je velice nízká, tj. $P('H'|'X') \rightarrow 0$
- pokud je v textu znak Q , je více než 99% pravděpodobnost, že bude následovat znak U . Přitom pravděpodobnost výskytu U je pouze 2%.
- vytvoříme-li vhodný model dat, který popisuje tuto skutečnost, jsme schopni poměrně efektivně kódovat anglicky psaný text

Model vyššího řádu

- Používat modely vyššího řádu se jeví jako velmi výhodné. V praxi však narazíme na řadu omezení a je proto nutné volit rozumný kompromis
 - s rostoucím řádem vzrůstají typicky paměťové nároky

Příklad: Předpokládejme 8-bitové kódy ASCII, pak velikost abecedy je 256 symbolů.

Nechť $N = 2$, pak existuje $256^N = 256^2 = 65536$ kontextů 2. řádu, $256^3 = 16\,777\,216$ kontextů 3. řádu a $256^4 = 4\,294\,967\,296$ kontextů 4. řádu.

- s rostoucím řádem vzrůstá počet výskytů, které jsme viděli pouze jednou nebo vůbec

Příklad: Nechť $N = 2$, uvažujme statický model. Po přečtení a analýze obrovského množství anglických textů jsme dosud nenašli trigram **qqz**, buňka odpovídající **qqz** v tabulce frekvencí trigramů bude obsahovat nulu. Pokud takový model použijeme a na vstupu se objeví **qqz**, budeme mít problém.

Problém s nulovými pravděpodobnostmi (zero frequency problem)

■ Entropický kodér se neumí vypořádat s nulovými pravděpodobnostmi

- Aritmetický kodér požaduje, aby všechny symboly měly nenulovou pravděpodobnost jinak bychom získali nekonečně dlouhou sekvenci bitů.
- Huffmanova metoda kombinuje dva symboly s nízkou pravděpodobností do jednoho s vyšší pravděpodobností. Když se zkombinují dva symboly a jeden z nich má nulovou pravděpodobnost, výsledný symbol bude mít stejnou pravděpodobnost jako nenulový → nejednoznačně dekodovatelný kód.

■ Možná řešení (žádné nemá teoretické opodstatnění)

- Použít **pseudopočet** – přidat kladnou hodnotu α ke každému počtu výskytů. Typicky $\alpha = 1$. Předpokládáme, že se každý symbol vyskytl alespoň α – krát. Celkový počet výskytu symbolů se zvýší. Tzn. každý i -tý ($i \geq 0$) nově nalezený symbol bude mít predikovanou pravděpodobnost $\alpha / (i + \alpha|\Sigma|)$.
- Přidat **escape symbol** před každý případ, kdy narazíme na první výskyt symbolu. Nový symbol je zakódován s uniformní pravděpodobností výskytu vzhledem k doposud nenalezeným symbolům Σ (tj. kódem pevné délky). Escape symbol používá předem stanovenou pravděpodobnost – typicky se přidá k celkovému počtu výskytů 1, která se rozdělí rovnoměrně mezi všechny symboly. Tím se přidělí velmi malá pravděpodobnost čemukoliv, co nebylo při analýze nalezeno.

Druhy modelů – I/II

■ Statický model

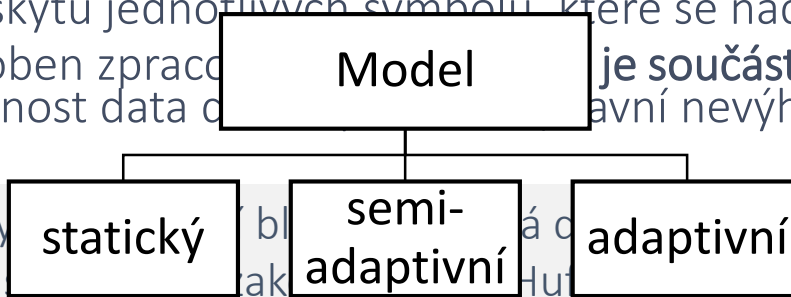
- fixní model, který **není odvozen z aktuálně zpracovávaného vstupního toku** a který je znám předem

Příklad: model četnosti výskytu symbolů (znaků, bigramů, trigramů) anglické abecedy získaný analýzou korpusu anglických textů

■ Semistatický / semiadaptivní model

- využívá se znalosti četnosti výskytu jednotlivých symbolů, které se nachází / nacházely ve vstupním toku
- semistatický model je přizpůsoben zpracování dat, které **je součástí výstupního toku** (tj. komprimovaných dat) tak, aby měl dekodér možnost data dekomprimovat. Hlavní nevýhoda tohoto přístupu zejména v případě komprese krátkých dat)

Příklad: Huffmanův kodér analyzuje vstupní tok a ukládá si do výstupního toku současně



pravděpodobnosti jednotlivých symbolů, uloží je do výstupního toku současně s komprimovanými daty. Hlavní nevýhoda tohoto přístupu zejména v případě komprese krátkých dat)

■ Adaptivní (dynamický) model

- model, který se mění v průběhu komprese;
- od určitého okamžiku se model stává funkcí již zpracované (zkomprimované) části vstupního toku a tudíž jej není zapotřebí ukládat do zkomprimovaného výstupu
- model není nutné ukládat na výstup, čímž ušetříme místo. Avšak z důvodů pomalé adaptace může tento model dávat horší ve výsledku horší kompresní poměr než statický / semistatický model

Příklad: adaptivního modelu nultého řádu: můžeme začít komprimovat data s využitím uniformního rozložení pravděpodobnosti a s každým zpracovaným symbolem distribuci poupravit na základě četnosti výskytů jednotlivých symbolů v již zpracovaném textu; nevýhoda: pomalá adaptace na vstupní data

Druhy modelů – II/II

■ Model s pevným řádem

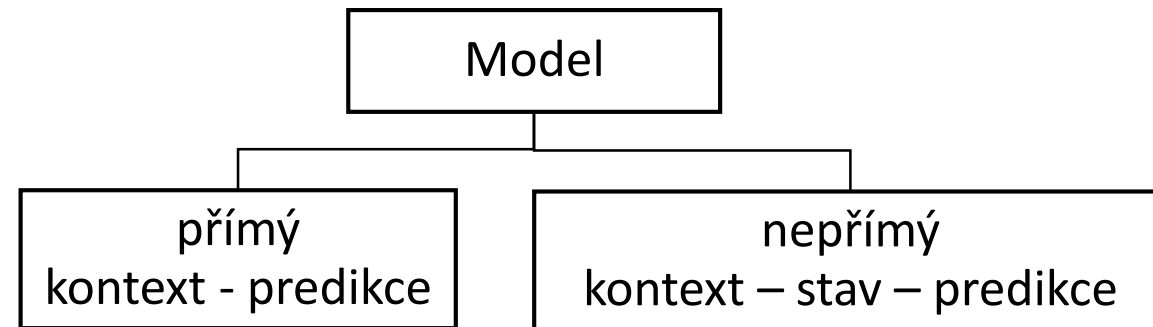
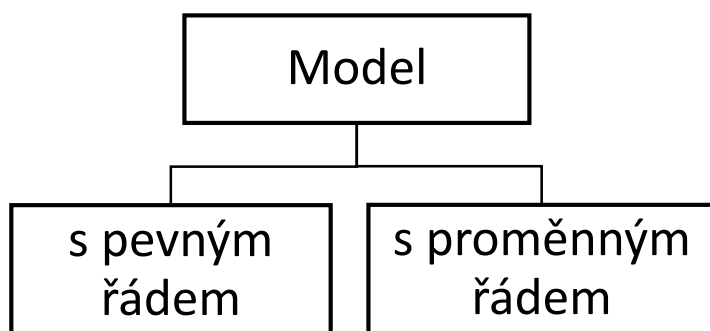
- Řád modelu je daný předem a v průběhu komprese se nemění.
- Model může být statický, semi-adaptivní nebo adaptivní
- Model může pracovat po bytech, bitech nebo kombinaci obou

Příklad: Adaptivní model s nultým řádem pracující po bytech je nejjednodušším z modelů kontextu.

■ Model s proměnným řádem

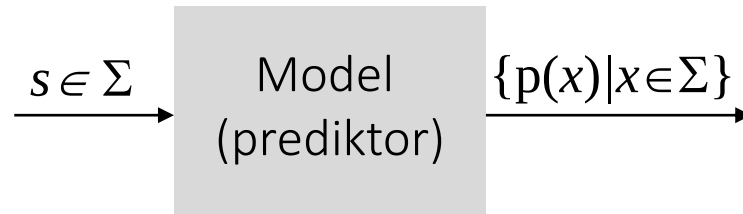
- Řád modelu se může měnit s každým načteným symbolem
- Model je typicky adaptivní

Příklad: Sběr statistik pro různé řády současně a následné použití nejdelšího shodného kontextu, o kterém něco víme.



Příklad: statický model nultého řádu pracující po znacích

- Příklad: uvažujme řetězec $S = \text{badada}$, $\Sigma = \{a, b, c, d\}$ a model nultého řádu.



- **Statický model:**

- jednotlivé symboly budou predikovány s předem danou pravděpodobností bez ohledu na četnost výskytu v S .

b	a	d	a	d	a
0.00142	0.00816	0.00425	0.00816	0.00425	0.00816

https://en.wikipedia.org/wiki/Letter_frequency

- **Semi-adaptivní model:**

- nejprve se analyzuje S , určí pravděpodobnosti a ty následně použijí

b	a	d	a	d	a
1/6	3/6	2/6	3/6	2/6	3/6
0.166	0.5	0.333	0.5	0.333	0.5

Příklad: statický model nultého řádu pracující po znacích

- Semi-adaptivní model:

b	a	d	a	d	a
1/6	3/6	2/6	3/6	2/6	3/6
0.166	0.5	0.333	0.5	0.333	0.5

- Teoretická délka výstupu:

$$- L = - \left(\log_2 \frac{1}{6} + \log_2 \frac{3}{6} + \log_2 \frac{2}{6} + \log_2 \frac{3}{6} + \log_2 \frac{2}{6} + \log_2 \frac{3}{6} \right) = 8.75 \text{ bitů}$$

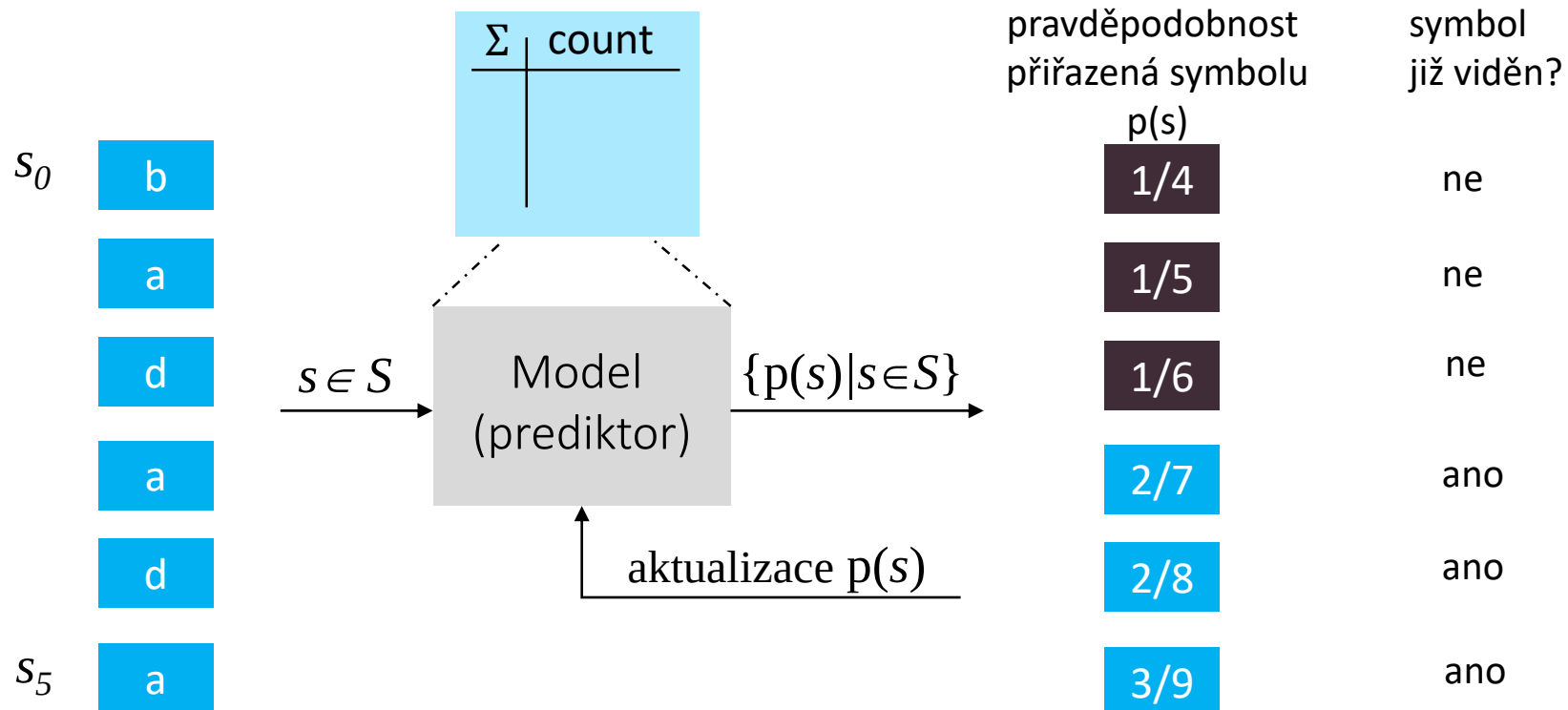
Adaptivní model nultého řádu pracující po znacích

- Princip: při dekódování i -tého znaku ($i \geq 0$) už známe četnosti znaků v $S[0..i-1]$. Tyto četnosti můžeme použít pro zakódování i -tého znaku.
 - Četnosti není nutné přenášet do dekodéru.
 - Model obsahuje tabulku, která každému znaku přiřazuje pravděpodobnost výskytu (stačí ukládat pouze počet výskytů daného znaku).
- S každým načteným znakem je nutné provést dva kroky:
 - vyhledání četnosti v tabulce a
 - aktualizaci tabulky
- Příklad (bez uvažování zero-frequency problem):
 - Předpokládejme, že zatím vstoupilo a bylo zakódováno 1217 symbolů a 34 z nich bylo písmeno q.
 - Je-li následující symbol q, dostane pravděpodobnost $34/1217$ a jeho počet se zvýší o 1.
 - Když se narazí na q příště, tj. v i -tém kroku, dostane přidělenou pravděpodobnost $35/i$.

$$p(s_i) = \frac{\text{count}(s_i)}{i}$$

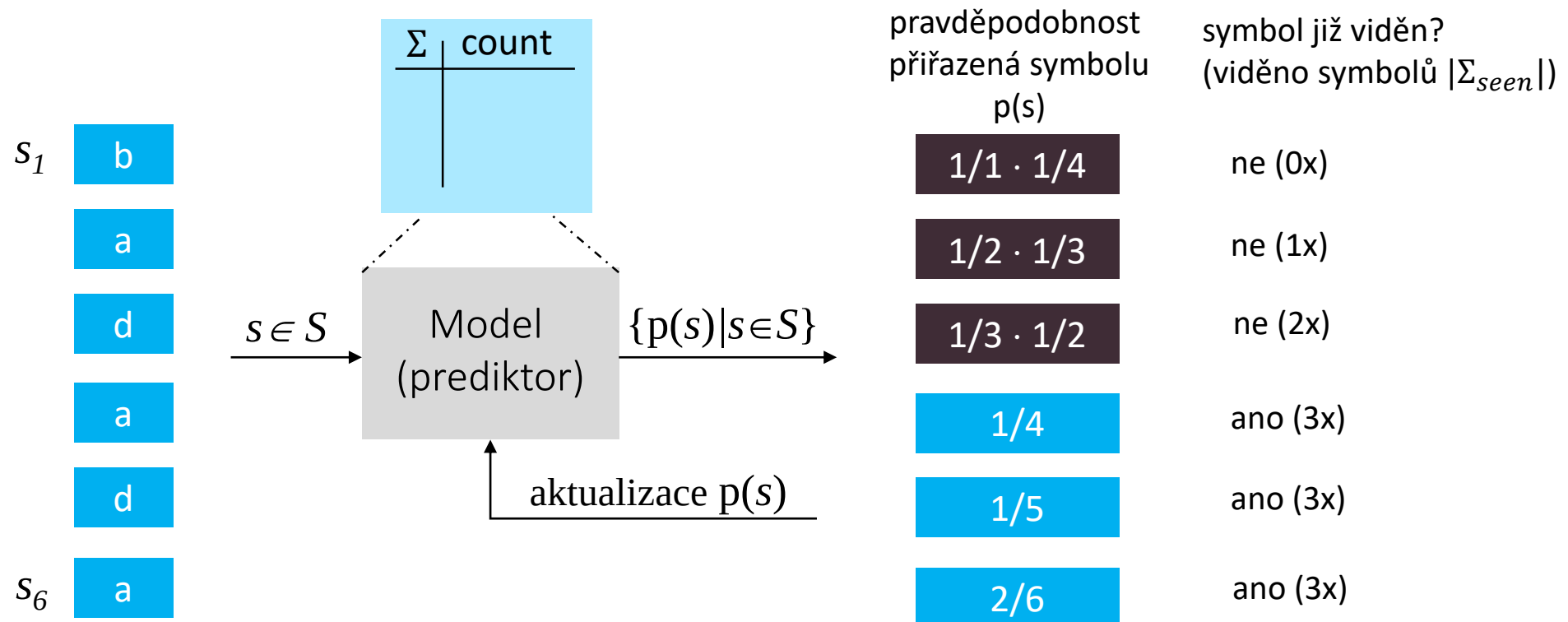
Příklad: model nultého řádu pracující po znacích

- Model: Adaptivní model nultého řádu využívající pseudopočet
 - $\alpha = 1$,
 - pro i -tý ($i \geq 0$) nově příchozí znak se použije $p(s_i) = \frac{1}{i+|\Sigma|}$, jinak $p(s_i) = \frac{\text{count}(s_i)+1}{i+|\Sigma|}$
- Př.: Uvažujme řetězec $S=\text{badada}$, $\Sigma = \{a, b, c, d\}$ a uvedený model.



Příklad: model nultého řádu pracující po znacích

- Model: Adaptivní model nultého řádu využívající escape symbol
 - $n_{escape} = 1$,
 - pro i -tý ($i \geq 1$) nově příchozí znak se použije $p(s_i) = \frac{1}{i} \cdot \frac{1}{|\Sigma \setminus \Sigma_{seen}|}$, jinak $p(s_i) = \frac{count(s_i)}{i}$
- Př.: Uvažujme řetězec $S=badada$, $\Sigma = \{a, b, c, d\}$ a uvedený model.



Očekávaná délka výstupu

- Adaptivní model nultého řádu využívající pseudopočet

b	a	d	a	d	a
$\frac{1}{4}$	$\frac{1}{5}$	$\frac{1}{6}$	$\frac{2}{7}$	$\frac{3}{8}$	$\frac{2}{9}$
0.25	0.2	0.166	0.285	0.375	0.222

$$L = -\left(\log_2 \frac{1}{4} + \log_2 \frac{1}{5} + \log_2 \frac{1}{6} + \log_2 \frac{2}{7} + \log_2 \frac{3}{8} + \log_2 \frac{2}{9}\right) = 12.3 \text{ bitů} = 2.05 \text{ bps}$$

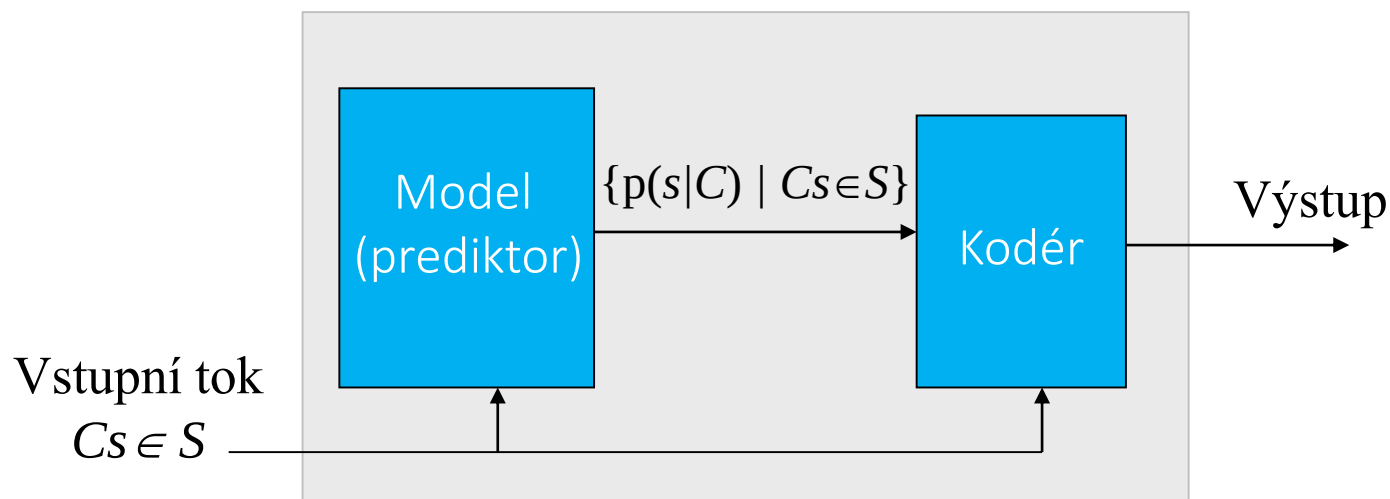
- Adaptivní model nultého řádu využívající escape symbol

b	a	d	a	d	a
$\frac{1}{4}$	$\frac{1}{6}$	$\frac{1}{6}$	$\frac{1}{4}$	$\frac{1}{5}$	$\frac{2}{6}$
0.25	0.166	0.166	0.25	0.2	0.333

$$L = -\left(\log_2 \frac{1}{4} + \log_2 \frac{1}{6} + \log_2 \frac{1}{6} + \log_2 \frac{1}{4} + \log_2 \frac{1}{5} + \log_2 \frac{2}{6}\right) = 13.1 \text{ bitů} = 2.18 \text{ bps}$$

Kontextové modely

- V praxi se typicky bere v úvahu pravděpodobnost výskytu symbolu v daném kontextu (digram, trigram, apod.), tj. model *uvažuje podmíněné rozd. pravděpodobnosti* $p(s|C)$
- Kontextový model
 - využívá se faktu, že použitím kontextu lze snížit entropii dalšího symbolu, což nám umožní zakódovat symbol pomocí méně bitů
 - **kontext je použit k predikci dalšího symbolu**, predikce a symbol se zašle do kodéru, který následně aktualizuje model podle symbolu, který právě zakódoval.



Kontextové modely – motivace

- Uvažujme na vstupu textový řetězec osmi znaků a kontext prvního řádu ($N=1$)

t h e t e x t

- Pokud bychom kódovali znak **h** samostatně, tak $p(\mathbf{h})=1/8$, tj. dostal by přiřazen kód délky 3 bity
- Pokud budeme uvažovat kontext řádu 1, pak s pravd. 1/2 se za **t** vyskytuje **h** nebo **e** $\Rightarrow$ k zakódování stačí pouze jeden bit
- Implementace: udržovat tabulku pravděpodobností pro každý kontext

t	$p(\mathbf{h})=1/2, p(\mathbf{e})=1/2$
h	$p(\mathbf{e})=1$
e	$p(\mathbf{' '})=1/2, p(\mathbf{x})=1/2$
	$p(\mathbf{t})=1$
x	$p(\mathbf{t})=1$

Kontextové modely – volba řádu

- Teoreticky čím je N větší, tím je lepší odhad pravděpodobnosti (predikce)
- Velké hodnoty N však mají řadu praktických nevýhod
 1. Počet možných kontextů roste exponenciálně s N => značné paměťové nároky
 2. Když zakódujeme symbol na základě 10 000 jej předcházejících symbolů, jak zakódujeme prvních 10 000 symbolů vstupního toku? Prosté kódy ASCII? => snížení celkové komprese.
 3. Velmi dlouhý kontext udržuje informaci o povaze starých dat avšak, vstupní řetězce typicky obsahují v různých částech rozdílná rozdělení symbolů. => Lepší komprese se proto dosáhne, když model přikládá menší význam informacím získaným ze starých dat a větší váhu čerstvým nedávným datům. Takového efektu dosáhneme krátkým kontextem.
- **Důsledek: v praxi se používají relativně krátké kontexty, v rozmezí od 2 do 10**
 - Každý praktický algoritmus vyžaduje pečlivě navrženou datovou strukturu, která umožňuje rychlé prohledávání a snadnou aktualizaci, přičemž udržuje mnoho tisíc symbolů a řetězců

Adaptivní model s pevným řádem (pracující po znacích)

■ Adaptivní model s pevným řádem (pracující po 8-bitových znacích):

- vstupem do modelu řádu N je posledních N bytů (symbolů, bitů), které ukazují do tabulky, která slouží k určení rozdělení pravděpodobnosti pro příští symbol.
- pro každý kontext je v tabulce modelu uchováván počet výskytů daného symbolu

```
int count[CONTEXT_SIZE][256] = {{0}};  
const int CONTEXT_SIZE = 1 << (N*8); // pro řád N
```

■ Kód kodéru

```
int context = 0; // inicializace (kontext C)
```

```
compress(char c)  
{  
    encode(count[context], c); // predikce a zakódování (aritm. kódér)  
    update(c); // aktualizace  
}
```

- Ve fázi aktualizace, kdy je znám predikovaný symbol se tabulka aktualizuje zvýšením pravděpodobnosti případu, že se opět objeví stejný kontext.

```
update(char c) { ++count[context][c]; context = (context << 8 | c) % CONTEXT_SIZE; }
```

Adaptivní model s pevným řádem (pracující po znacích)

■ Adaptivní model s pevným řádem (pracující po 8-bitových znacích):

- vstupem do modelu řádu N je posledních N bytů (symbolů, bitů), které ukazují do tabulky, která slouží k určení rozdělení pravděpodobnosti pro příští symbol.
- pro každý kontext je v tabulce modelu uchováván počet výskytů daného symbolu

```
int count[CONTEXT_SIZE][256] = {{0}};  
const int CONTEXT_SIZE = 1 << (N*8); // pro řád N
```

■ Kód dekodéru

```
int context = 0; // inicializace (kontext C)
```

```
char decompress()  
{ char c = decode(count[context]);  
  update(c);  
  return c;  
}
```

```
update(char c) { ++count[context][c]; context = (context << 8 | c) % CONTEXT_SIZE; }
```



Adaptivní model s pevným řádem (pracující po znacích)

■ Princip modelu

- Model obsahuje tabulku kontextů *count[]*. V případě bytů má tabulka kontextů $N \times 256$ položek.
- Predikcí pro každý symbol je počet pro tento symbol podělený celkovým počtem výskytů symbolu v daném kontextu.
- Aritmetický kódér kóduje v každém okamžiku jeden byte *c*, do aritmetického kódéru se předá **pole počtů** pro daný kontext. Při kompresi se předá také kódovaný byte. Při dekompresi se vrací dekódovaný byte.
- Uvedený adaptivní model kontextu řádu N defacto udržuje 256 oddělených modelů nultého řádu. Je-li *S* vstupní tok, pak každý *i*-tý symbol si toku je zakódován pomocí modelu pro kontext $S[i-n:i-1]$.

■ Praktické problémy, které je nutné řešit:

- Vhodná volba řádu adaptivního modelu: příliš malé N nedává dobrý kompresní poměr. Příliš velké N vede na špatný kompresní poměr na začátku. => prediction by partial matching (PPM)
- Omezený počet bitů jednotlivých položek tabulky může způsobit přetečení => potřeba škálování
- Velikost tabulky roste exponenciálně s N => nahradit kontext výsledkem hash funkce

```
const int CONTEXT_SIZE = 1 << (N*k); // k je počet bitů hash na jeden byte, k = 5
```

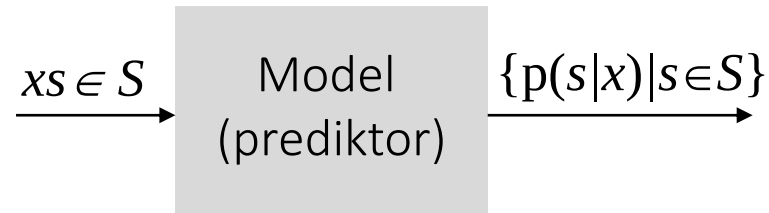
```
...
```

```
context = (context * (3 << k) + c) % CONTEXT_SIZE; // update paměti kontextu
```



Příklad: model prvního řádu pracující po znacích

- Příklad: uvažujme řetězec $S = \text{badadabada}$, $\Sigma = \{a, b, c, d\}$ a model prvního řádu.



- Semi-adaptivní model:

- nejprve se analyzuje S , určí všechny podmíněné pravděpodobnosti

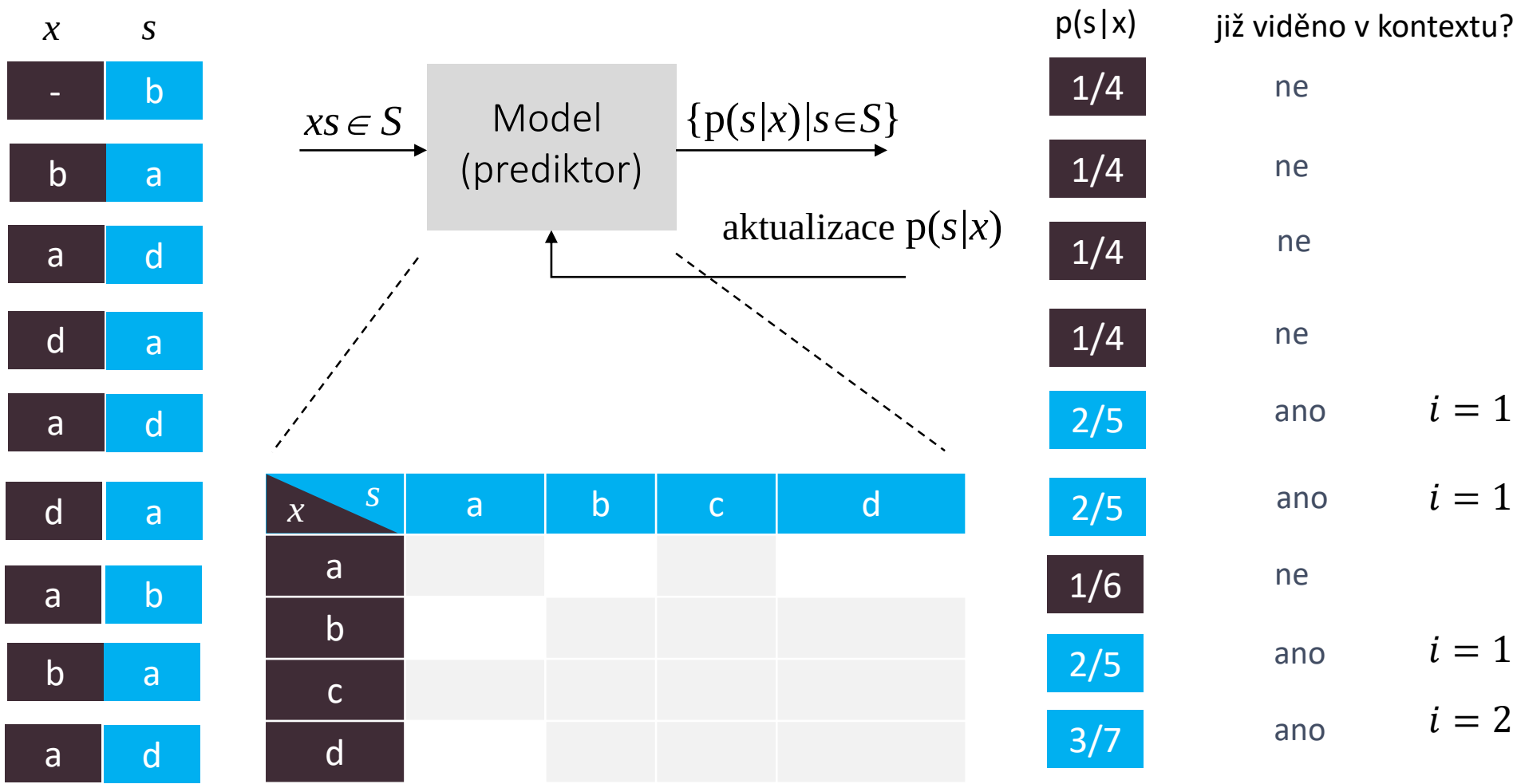
$x \backslash s$	a	b	c	d
a	0/4	1/4	0/4	3/4
b	2/2	0/2	0/2	0/2
c	0/0	0/0	0/0	0/0
d	3/3	0/3	0/3	0/3

- následně model použijeme (pozor na první znak)

b	a	d	a	d	a	b	a	d	a
1/4	2/2	3/4	3/3	3/4	3/3	1/4	2/2	3/4	3/3

Příklad: model prvního řádu pracující po znacích

- Model: Adaptivní model využívající pseudopočet
 - $\alpha = 1$, pro i -tý ($i \geq 0$) nově přichází znak kontextu x se použije $p(s_i|x) = \frac{1}{i+|\Sigma|}$, jinak $p(s_i|x) = \frac{\text{count}(s_i|x)+1}{i+|\Sigma|}$
- Příklad: uvažujme řetězec $S=\text{badadabada}$, $\Sigma = \{a, b, c, d\}$ a model prvního řádu.



Příklad: $s = \text{badada}$, $|s| = 6$

- Adaptivní model nultého řádu
 - pseudopočet $L = 2.05$ bps, escape symbol $L = 2.18$ bps
- Adaptivní model prvního řádu využívající pseudopočet

b	a	d	a	d	a
$p(b)$	$p(a b)$	$p(d a)$	$p(a d)$	$p(d a)$	$p(a d)$
$\frac{1}{4}$	$\frac{1}{4}$	$\frac{1}{4}$	$\frac{1}{4}$	$\frac{2}{5}$	$\frac{2}{5}$
0.25	0.25	0.25	0.25	0.4	0.4

$$L = -\left(\log_2 \frac{1}{4} + \log_2 \frac{1}{4} + \log_2 \frac{1}{4} + \log_2 \frac{1}{4} + \log_2 \frac{2}{5} + \log_2 \frac{2}{5}\right) = 10.64 \text{ bitů} = 1.77 \text{ bps}$$

- Adaptivní model prvního řádu využívající escape symbol

b	a	d	a	d	a
$p(E)p(b)$	$p(E b)p(a b)$	$p(E a)p(d a)$	$p(E d)p(a d)$	$p(d a)$	$p(a d)$
$1 \cdot \frac{1}{4}$	$1 \cdot \frac{1}{4}$	$1 \cdot \frac{1}{4}$	$1 \cdot \frac{1}{4}$	$\frac{1}{2}$	$\frac{1}{2}$
0.25	0.25	0.25	0.25	0.5	0.5

$$L = -\left(\log_2 \frac{1}{4} + \log_2 \frac{1}{4} + \log_2 \frac{1}{4} + \log_2 \frac{1}{4} + \log_2 \frac{1}{2} + \log_2 \frac{1}{2}\right) = 10.0 \text{ bitů} = 1.67 \text{ bps}$$

Příklad: $s = \text{badadadadadadabada}$, $|s| = 20$

■ Adaptivní model nultého řádu

- pseudopočet $L = 33.77$ bitů $= 1.69$ bps
- escape symbol $L = 34.89$ bitů $= 1.74$ bps

■ Adaptivní model prvního řádu

- pseudopočet $L = 23.64$ bitů $= 1.18$ bps
- escape symbol $L = 19.75$ bitů $= 0.99$ bps

■ $p(b)=2/20$, $p(a)=10/20$, $p(d)=8/20$

- pomocí HC nejlépe $L = 10 \times 1 + 8 \times 2 + 2 \times 2 = 30$ bitů

Adaptivní model s pevným řádem pracující po bitech

■ Adaptivní model s pevným řádem (pracující bitech):

- založen na myšlence kódovat bit po bitu s využitím již zakódovaných bitů aktuálního symbolu jako doplňujícího kontextu bytový kontext + bitový kontext => predikce
- principiálně musí být pro každou dvojici kontext + bitový kontext (hodnota 1 – 255) v tabulce modelu uchováván počet výskytů jedničkového bitu a celkový počet bitů. Predikcí potom je poměr těchto dvou hodnot. To je však nevýhodné (operace dělení je pomalá). Řešení: uchovávat přímo predikci

```
struct Model {  
    double prediction; // určuje pravd. (hodnota mezi 0 and 1), že další bit bude 1  
    int count; // počítadlo aktualizací  
    Model(): prediction(0.5), count(0) {}  
};  
  
Model model[CONTEXT_SIZE][256];  
const int CONTEXT_SIZE = 1 << (N*8); // řád modelu je N
```

- počítadlo aktualizací slouží k přizpůsobení rychlosti aktualizace hodnot. Pokud počítadlo omezíme max. hodnotou, dostaneme adaptivní model. Jinak dostáváme model stacionární.



Adaptivní model s pevným řádem pracující po bitech

■ Adaptivní model s pevným řádem (pracující bitech):

- během aktualizace se určí chyba predikce ($\text{bit} - \text{m.prediction}$) a upraví se m.prediction v opačném poměru vzhledem k počítadlu. Současně se inkrementuje počítadlo dokud nenarazíme na LIMIT. V tom okamžiku se model přepíná ze stacionárního na adaptivní.

```
const double DELTA = 0.5; //poč. stav
const int LIMIT = 255;
void update(int bit, Model& m) {
    if (m.count < LIMIT) ++m.count;
    m.prediction += (bit - m.prediction) / (m.count + DELTA);
}
```

- DELTA and LIMIT jsou parametry, kterými lze ovlivnit chování modelu. Velká hodnota LIMIT je vhodná pro stacionární data. Menší hodnota je vhodnější pro smíšená data. Hodnota DELTA je kritická pouze pokud velikost kódovaných dat je malá. Pokud je DELTA 1, pak sekvence nulových bitů vede na predikci $1/2, 1/4, 1/6, 1/8, 1/10$.
- V praxi se používá fixed point reprezentace a dělení se nahrazuje lookup tabulkou

Adaptivní model s pevným řádem pracující po bitech

■ Adaptivní model s pevným řádem (pracující bitech):

- vstupní tok se načítá po 8-bitových znacích, každý znak je následně kódován po bitech počínaje MSB a konče LSB. Již zakódované bity aktuálního znaku tvoří kontext. Ten nabývá hodnot 1 až 255.

```
void compress(char c) {  
    for (int i=7; i>=0; --i) {  
        int bit_context = c+256 >> i+1; // bitový kontext  
        int bit = (c >> i) & 1;  
        encode(bit, model[context][bit_context].prediction);  
        update(bit, model[context][bit_context]);  
    }  
    context = (context << 8 | c) % CONTEXT_SIZE;  
}
```

- Příklad: pokud $c = 00011100$, pak kontext nabývá postupně hodnot 1, 10, 100, 1000, 10001, 100011, 1000111, 10001110

Adaptivní model s pevným řádem pracující po bitech

- Adaptivní model s pevným řádem (pracující bitech):
 - dekomprese probíhá obdobně jako komprese

```
char decompress() {  
    int c; // bitový kontext  
    int bit;  
    for (c = 1; c < 256; c = c << 1 + bit) {  
        bit = decode(model[context][c].prediction);  
        update(bit, model[context][c]);  
    }  
    c -= 256; // dekódovaný znak  
    context = (context << 8 | c) % CONTEXT_SIZE;  
    return c;  
}
```

Modely s proměnným řádem

- Vyrůstající řád modelu pevného řádu vede na zvýšení kompresní účinnosti až do určitého bodu (pro korpus Calgary řád 3).
 - Od tohoto bodu se komprese zhoršuje, protože mnoho kontextů vyššího řádu se vidí poprvé a není tak možné efektivně dělat predikci.
- Jednou z možností, jak problém překonat, je přejít na model s **proměnným řádem**
 - Sbírat statistiky pro různé řády současně a pak použít nejdelší shodný kontext, o kterém něco víme.
- Tento princip je použit u následujících metod:
 - PPM (Prediction by Partial Matching) pro predikce na bytové úrovni
 - DMC (Dynamic Markov Compression) pro predikce na bitové úrovni
 - CTW (Context Tree Weighting) kombinuje oba přístupy

PPM (prediction by partial matching)

- PPM (Cleary a Witten 1984) je nejznámější kontextovou metodou vyššího řádu, která používá řetězec symbolů v nekomprimovaném toku dat k predikci dalšího symbolu.
- Myšlenkou PPM je **vyhnout se nulovým pravděpodobnostem výskytu** tak, že snížíme řád kontextu
 - PPM se snaží používat menší a menší části kontextu C .
 - V případě, kdy se nepodařilo nalézt symbol S pro daný kontext C řádu N vyšle PPM kódér speciální značku, která způsobí přepnutí kódéru i dekodéru na řád $N - 1$. Následně se pokusí nalézt kratší řetězec. Pokud ani tehdy neuspěje, zkusí se opětovně snížit řád. Pokud se nenalezne S ani pro kontext řádu 0, pak je nutné zapsat na výstup nekomprimovaný znak S .
- Místo toho abychom používali jeden model řádu N , používáme defacto **sekvenci N modelů**, které nám pomohou určit podmíněnou pravděpodobnost
 - $p(\text{symbol} \mid \text{předchozích } N \text{ symbolů})$, $p(\text{symbol} \mid \text{předchozích } N-1 \text{ symbolů})$, ..., $p(\text{symbol} \mid \text{předchozí symbol})$, $p(\text{symbol})$

Varianta PPMC (Mofat, 1990)

- Způsob přidělení pravděpodobnosti symbolu změny (ESC) může být implementován různě.

- V případě PPMC se každý kontext uloží do tabulky do samostatné skupiny včetně symbolu ESC generovaného z důvodu snížení řádu kontextu.

- Pravděpodobnost výskytu symbolu $s \in \Sigma$ v daném kontextu $p(s|context) = \frac{f_s}{t+k}$,

pravděpodobnost ESC symbolu $p(ESC|context) = \frac{k}{t+k}$

- f_s je počet výskytů symbolu s v daném kontextu
- t je celkový počet symbolů viděných doposud v daném kontextu
- k je počet různých symbolů viděných doposud v daném kontextu

- **Algoritmus**

- je-li v modelu $p(s|context)$, použije se tato pravděpodobnost, jinak
- se pravděpodobnost odhadne tak, že se vyšle $p(ESC|context)$ a sníží se řád kontextu.
- Předchozí kroky se opakují tak dlouho, než je dosažen řád -1, kdy je použito uniformní rozdělení pravděpodobnosti.

'u'|'th'

$$p(ESC|'th') = 6/101 = 0.059$$

$$p('u'|'h') = 0.009$$

Order	2	1	0	-1
Context	'th'	'h'	empty	empty
Symbols				
'a'	8	33	226	1
'e'	51	110	362	1
'i'	22	24	188	1
'o'	7	16	248	1
sp	6	14	781	1
'.'	1	1	16	1
'u'	0	2	84	1
Total (t)	95	200	1,905	

Varianta PPMC (Mofat, 1990)

- Uvažujme řetězec $S = \text{assanissimassa}$, $\Sigma = \{a, i, n, m, s\}$, který již byl zakódován a kontextový model druhého řádu. Aktuální kontext „sa“ a další symbol je „s“
 - „sa“ následované „s“ se zatím nevyskytlo, vyšleme do aritmetického kodéru symbol ESC spolu s pravděpodobností $1/2$, kterou predikuje kontext „sa“ řádu 2. K zakódování ESC je zapotřebí 1 bit.
 - Po přepnutí na řád 1 se aktuální kontext změní na „a“, přičemž „a“ následované „s“ bylo viděno a má nyní přidělenou pravděpodobnost $2/5$. Proto se s pošle do aritmetického kodéru k zakódování s pravděpodobností $2/5$, což produkuje dalších 1,32 bitů.
 - K zakódování „s“ je tedy zapotřebí celkem $1 + 1,32 = 2,32$ bitů.
- Zavedením tzv. vyloučení (lazy exclusion) je možné zvýšit kompresní poměr
 - Když kodér PPM přepne z vyššího řádu na nižší, může použít informace ve vyšším řádu k vyloučení některých případů řádu nižšího, o kterých se ví, že jsou nemožné.
 - Skutečnost, že „sa“ se již vyskytlo, následované „n“ implikuje, že aktuální symbol nemůže být „n“ (kdyby bylo, bylo by zakódováno v řádu 2).
 - Kodér tak může vyloučit případ „a“ následovaného „n“ v kontextech řádu 1 (můžeme říct, že není zapotřebí rezervovat prostor pro pravděpodobnost tohoto případu, protože není možný). Tím se sníží celková pravděpodobnost skupiny řádu 1 „a→“ z 5 na 4, což zvýší pravděpodobnost, přidělenou „s“ z $2/5$ na $2/4$.
 - Na základě naší znalosti z řádu 2 se může „s“ nyní zakódovat pomocí $-\log_2(2/4) = 1$ bitu, namísto 1,32 bitů (to dá celkem 2 bity, protože ESC rovněž potřebuje 1 bit).

$$\begin{array}{l} n \mid sa \quad 1 \quad 1/2 \\ \text{ESC} \mid sa \quad 1 \quad 1/2 \end{array}$$

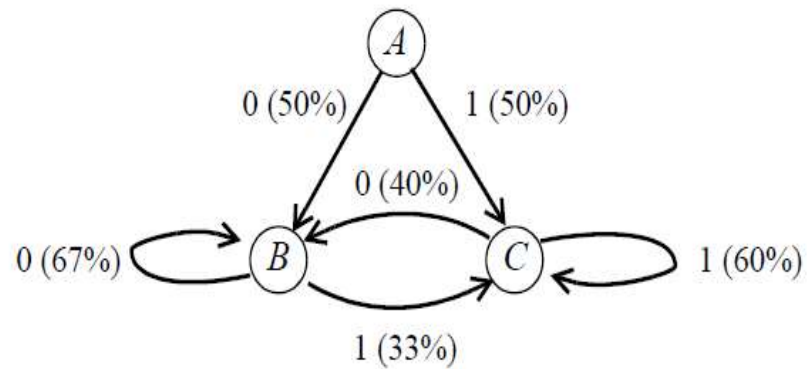
$$\begin{array}{l} s \mid a \quad 2 \quad 2/5 \\ n \mid a \quad 1 \quad 1/5 \\ \text{ESC} \mid a \quad 2 \quad 2/5 \end{array}$$

$$\begin{array}{l} s \mid a \quad 2 \quad 2/4 \\ \hline n \mid a \quad 1 \quad 1/5 \\ \text{ESC} \mid a \quad 2 \quad 2/4 \end{array}$$



DMC (Dynamic Markov Compression)

- Metoda DMC je adaptivní metoda (Cormack, Horspool 1987) založená na **dynamické konstrukci Markovova modelu**. Používá se přímý kontextový model, který je konstruován na základě dat, která přichází do kodéru.
- Příklad: Markovův model prvního řádu ($p(x|y)$ kde $x, y \in \Sigma$) pro abecedu tvořenou dvěma symboly $\Sigma = \{0,1\}$



0 is followed by 0 with probability $2/3$
0 is followed by 1 with probability $1/3$
1 is followed by 0 with probability $2/5$
1 is followed by 1 with probability $3/5$

- Markovův model je acyklický orientovaný graf, jehož hranám jsou přiřazeny pravděpodobnosti.
- Dynamickou tvorbu modelu lze rozdělit na dvě podúlohy:
 - přiřazení pravděpodobnosti hranám
 - tvorba stavového grafu

DMC – princip přiřazení pravděpodobnosti hranám

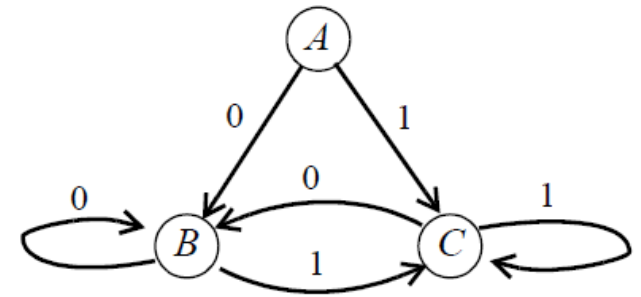
■ Princip přiřazení pravděpodobnosti hranám

- Předpokládejme, že máme k dispozici stavový graf obsahující jednotlivé přechody.
- Přiřazení pravděpodobnosti můžeme pak založit na počítadle provedení přechodů.
- U každého stavu můžeme mít dvě počítadla:
 - n_0 je počet provedení přechodu 0,
 - n_1 počet provedení přechodu 1.

Potom pravd. přechodu můžeme určit jako

$$\text{Prob} \{ \text{digit} = 0 \mid \text{current state} = A \} = n_0 / (n_0 + n_1)$$

$$\text{Prob} \{ \text{digit} = 1 \mid \text{current state} = A \} = n_1 / (n_0 + n_1)$$



- Abychom se vyhnuli nulovým pravděpodobnostem lze aplikovat pseudopočet (použít malou aditivní konstantu c)

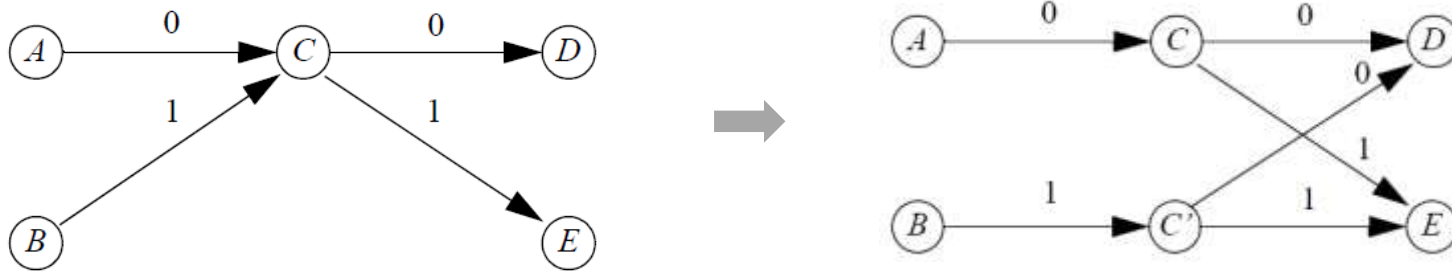
$$\text{Prob} \{ \text{digit} = 0 \mid \text{current state} = A \} = (n_0 + c) / (n_0 + n_1 + 2c)$$

$$\text{Prob} \{ \text{digit} = 1 \mid \text{current state} = A \} = (n_1 + c) / (n_0 + n_1 + 2c)$$

DMC – princip tvorby stavového grafu

■ Princip tvorby stavového grafu

- stavový graf je založen na tzv. **klonování uzlů**, jehož cílem je modifikovat strukturu tak, aby se automaticky naučila modelovat kontext
- Příklad: předpokládejme, že jsme v určitém bodě zkonstruovali následující graf sestávající z pěti stavů



- Model zachycuje situaci, že je vysoce pravděpodobné, že další stav bude D nebo E. Pokud však vstoupíme do stavu C, tak model kompletně ztratí přehled o kontextu (jak jsme do stavu vstoupili) a přitom může existovat vysoká korelace mezi tím, jak jsme vstoupili do C (jestli A a D) a tím, jak se rozhodneme dále (B a E).
- Klonování stavu C nám **umožňuje modelovat korelaci**. Klonování se provádí tehdy, pokud $\#(A \rightarrow C) > T1$ a $\#(* \rightarrow C \text{ kromě } A \rightarrow C) > T2$, přičemž $\#(X)$ je počet provedení přechodu X, T1 a T2 je práh, A je aktuální stav a C je stav, do kterého se má přejít.
 - Prahové konstanty T1 a T2 umožňují určit, jak často bude docházet ke klonování.
 - Příliš časté klonování vede na větší paměťové nároky a model se stává více citlivý na fluktuace (stavy budou navštíveny s menší pravděpodobností). Obvykle se oba parametry volí stejně (2 – 256).

DMC – princip tvorby stavového grafu

■ Princip tvorby stavového grafu

- **DMC se inicializuje** modelem, který je v případě dvouprvkové abecedy $\Sigma = \{0,1\}$ tvořen jedním stavem. Oba přechody jsou inicializovány na stejnou pravděpodobnost $n_0 = n_1 = 0$. Model se klonováním postupně rozšiřuje a zpřesňuje.



- Klonování vede k nárůstu paměti, je proto nutné zvolit nějakou omezující podmínku. Typicky dochází k **reinicializaci modelu** při překročení maximálního dovoleného počtu uzlů.
 - Reset modelu a nahrazení jedním uzlem (příliš drastické)
 - Reset modelu, a jeho nahrazení modelem, který vznikne průchodem bufferu zachycujícího posledních k zpracovaných bytů, což vede na model s relativně malým počtem stavů

DMC – implementace

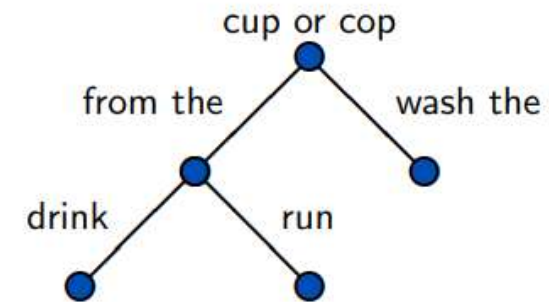
- DMC je vhodné implementovat pomocí tabulky. Není nutné vyhledávat kontext, ale stačí si pamatovat ukazatel na aktuální kontext a při aktualizaci následovat odkazy vedoucí z aktuální položky.
 - Indexem do tabulky je bitový kontext.
 - Položka tabulky obsahuje
 - počítadla n_0 a n_1 .
 - dva ukazatele, které ukazují do tabulky na kontext, který získáme přidáním nuly nebo jedničky zprava a (případným) odebráním jednoho symbolu zleva.
- **Kompresní účinnost**
 - DMC dává co se kompresního poměru týče podobné výsledky jako PPM v případě textu. Pokud komprimujeme binární data, DMC dosahuje lepšího kompresního poměru.

Kontextový strom (Context Tree)

- Jinou možností, jak **modelovat kontext proměnné délky** je použít kontextový strom (Context Tree) uzly odpovídají stavům a hrany přechodům. Hranám je přidělena pravděpodobnost přechodu.

- **Příklad kontextového stromu pracujícího na úrovni slov:**

- Pokud jsme slyšeli "from the" a chceme se rozhodnout zda-li další slovo je "cup" nebo "cop" pomůže nám znát i předchozí slovo, jestli se jedná o "drink from the" nebo "run from the".
- V jiném případě je postačující znát pouze „wash the“ pro jednoznačnou odpověď.

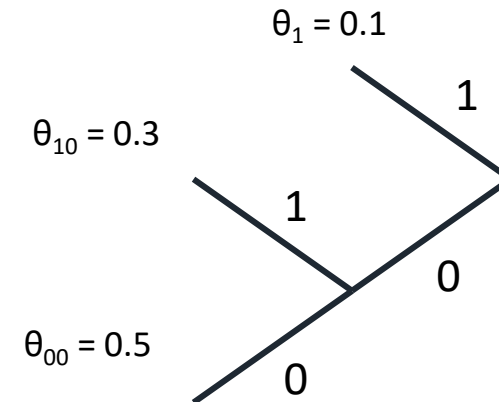


- My budeme uvažovat abecedu obsahující dva symboly $\Sigma=\{0,1\}$.
 - zajímá nás, bude-li následovat 1 nebo 0

Kontextový strom (Context Tree)

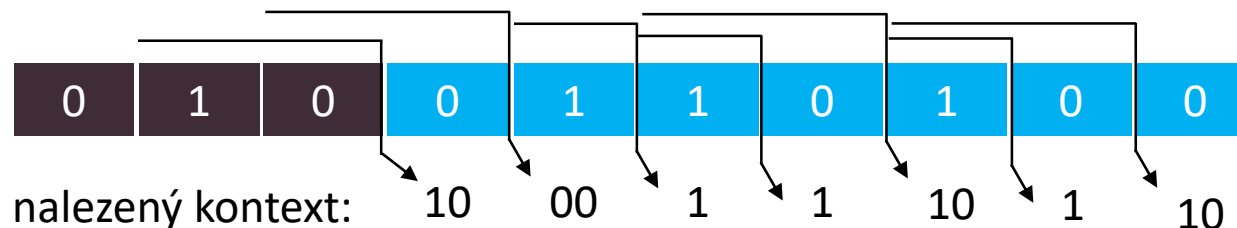
- Uvažujme následující kontextový strom modelující kontext maximální délky 3 a obsahující tři stavy $T = \{00, 10, 1\}$

- $P(x_t = 1 | \dots x_{t-1} = 1) = \theta_1$
- $P(x_t = 1 | \dots x_{t-2} = 0, x_{t-1} = 0) = \theta_{00}$
- $P(x_t = 1 | \dots x_{t-2} = 1, x_{t-1} = 0) = \theta_{10}$



Type equation here.

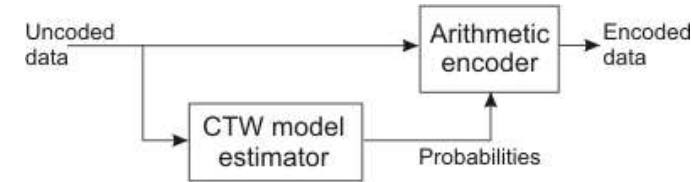
- Jaká je pravděpodobnost, že zdroj vygeneruje sekvenci 0110100 uvažujeme-li kontext 010 a je-li dán T uvedený výše?



$$-\log_2 0.0059535 = 7.392$$

$$P(0110100 | \dots 010) = (1 - \theta_{10}) \theta_{00} \theta_1 (1 - \theta_1) \theta_{10} (1 - \theta_1) (1 - \theta_{10}) = 0.0059535$$

Context Tree Weighting



- CTW (Willems, Shtarkov, Tjalkens, 1995) používá kontextový strom T_D pro jehož každý uzel platí, že
 - reprezentuje binární řetězec s , jehož délka je nanejvýše D .
 - má nanejvýše dva potomky. Uzel s tvoří rodiče uzlu $0s$ a $1s$.
 - má přiřazeny dvě počítadla, $a_s \geq 0$ (počet případů, kdy za s následovala 0) a $b_s \geq 0$ (počet případů, kdy za s následovala 1). Pro počítadla přiřazená potomkům uzlu s platí, že $a_{0s} + a_{1s} = a_s$ a $b_{0s} + b_{1s} = b_s$
- CTW používá k určení pravděpodobnosti výskytu dané sekvence rekurzivní vzorec, který začíná v listu a pokračuje směrem ke kořeni.
 - pro uzel s , který je listovým uzlem, platí, že $P_s = P_{kt}(a_s, b_s)$
 - pro uzel s , který není listovým uzlem, platí, že $P_s = 1/2 (P_{kt}(a_s, b_s) + P_{0s}P_{1s})$
- Odhad P_{kt} se počítá pomocí KT estimátoru (Krichevsky-Trofimov, 1981)
 - $P_{kt}(0,0) = 1$
 - $P_{kt}(a+1, b) = \frac{a+\frac{1}{2}}{a+b+1} \cdot P_{kt}(a, b)$
 - $P_{kt}(a, b+1) = \frac{b+\frac{1}{2}}{a+b+1} \cdot P_{kt}(a, b)$

```

double Pkt(int a, int b):
    if (a>0):
        return (a-0.5)/(a+b) * Pkt(a-1, b)
    elif (b>0):
        return (b-0.5)/(a+b) * Pkt(a, b-1)
    return 1
  
```

Context Tree Weighting

■ CTW je založeno na myšlence

- vytvořit prediktor, který je směsicí všech modelů kontextu omezených D .
- využít schopnosti kontextového stromu T_D modelovat souběžně všechny existující kontexty od nultého řádu až do řádu D .

■ Váhování jednotlivých modelů je založeno na dvojici hodnot (a,b) .

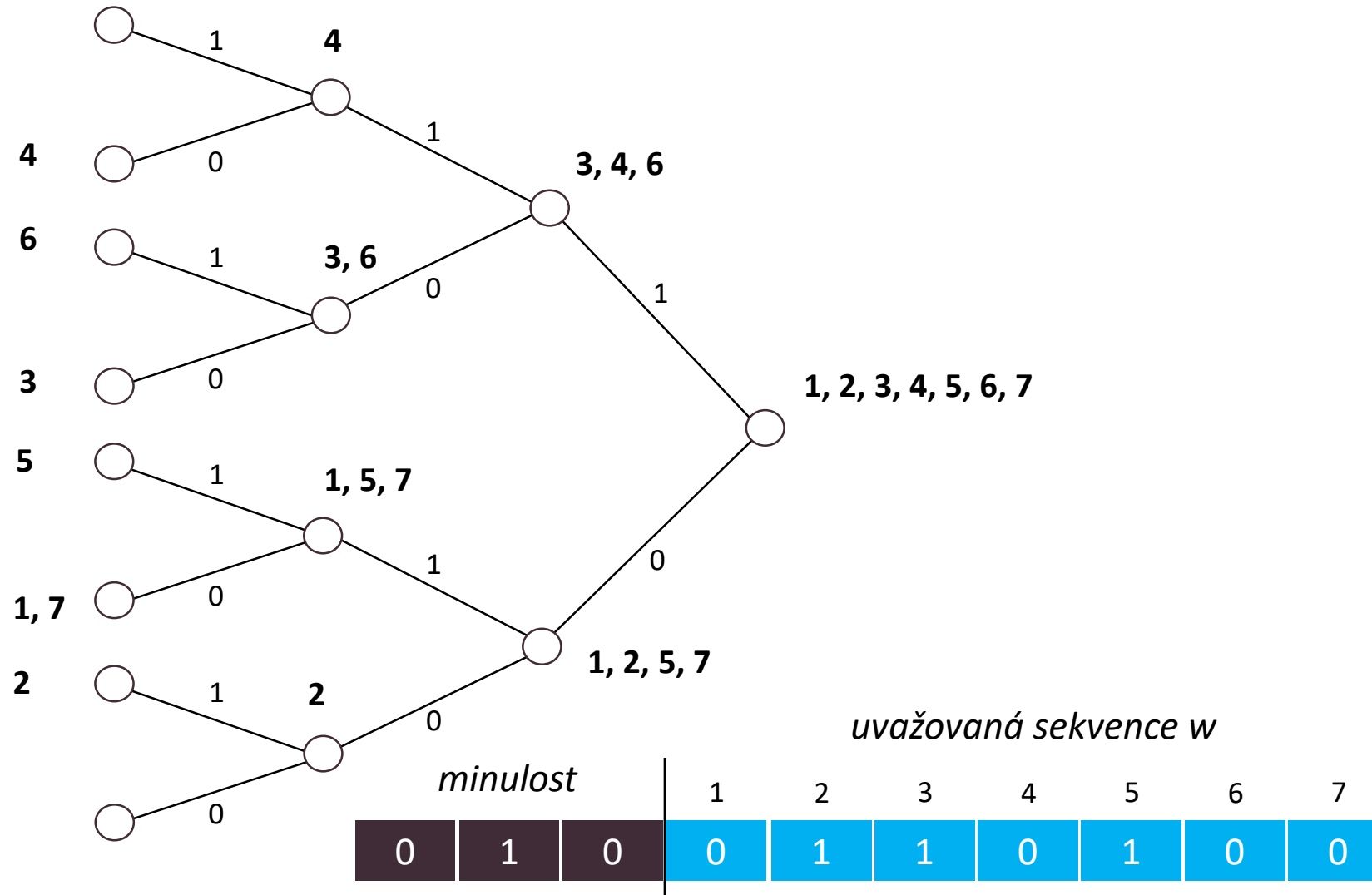
- Výsledná pravděpodobnost je vždy násobkem mocniny dvou a je možné ji tím pádem efektivně zakódovat (CTW se blíží teoretické optimální efektivitě).

■ Princip práce se stromem

- začíná inicializací stromu max. hloubky D .
- pro každý načtený vstupní bit se provede určení pravděpodobnosti výskytu řetězce, který se skládá z kontextu řádu D zprava doplněného o právě načtený bit
- následně se provede aktualizace stromu. Poměrně jednoduchá operace, která vyžaduje pouze aktualizaci hodnot přiřazeným uzlům ležícím na jedné konkrétní cestě dané kontextem. Díky rekurzivní definici KT není potřeba rekurzivní přepočítání.

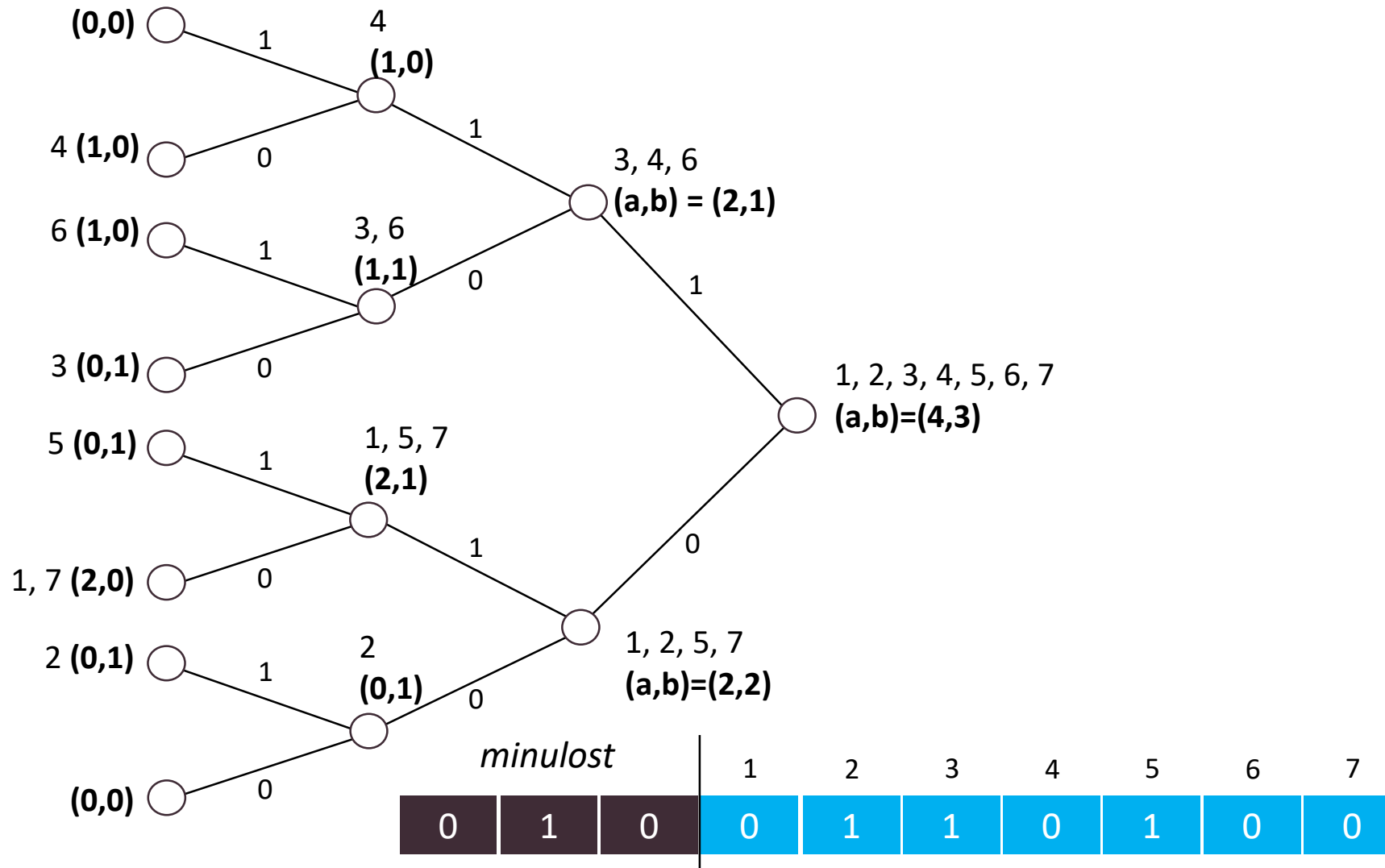
Princip určení pravděpodobnosti pomocí CTW

- Uvažujme váhovaný CT T_3 (tj. $D = 3$) pro sekvenci $h_i^T = 0110100$ a $x_{i-D}^0 = \dots 010$.
- 1) Od kořene postupně sestrojíme pro sekvenci h_i^T všechny kontexty délky 1 až D



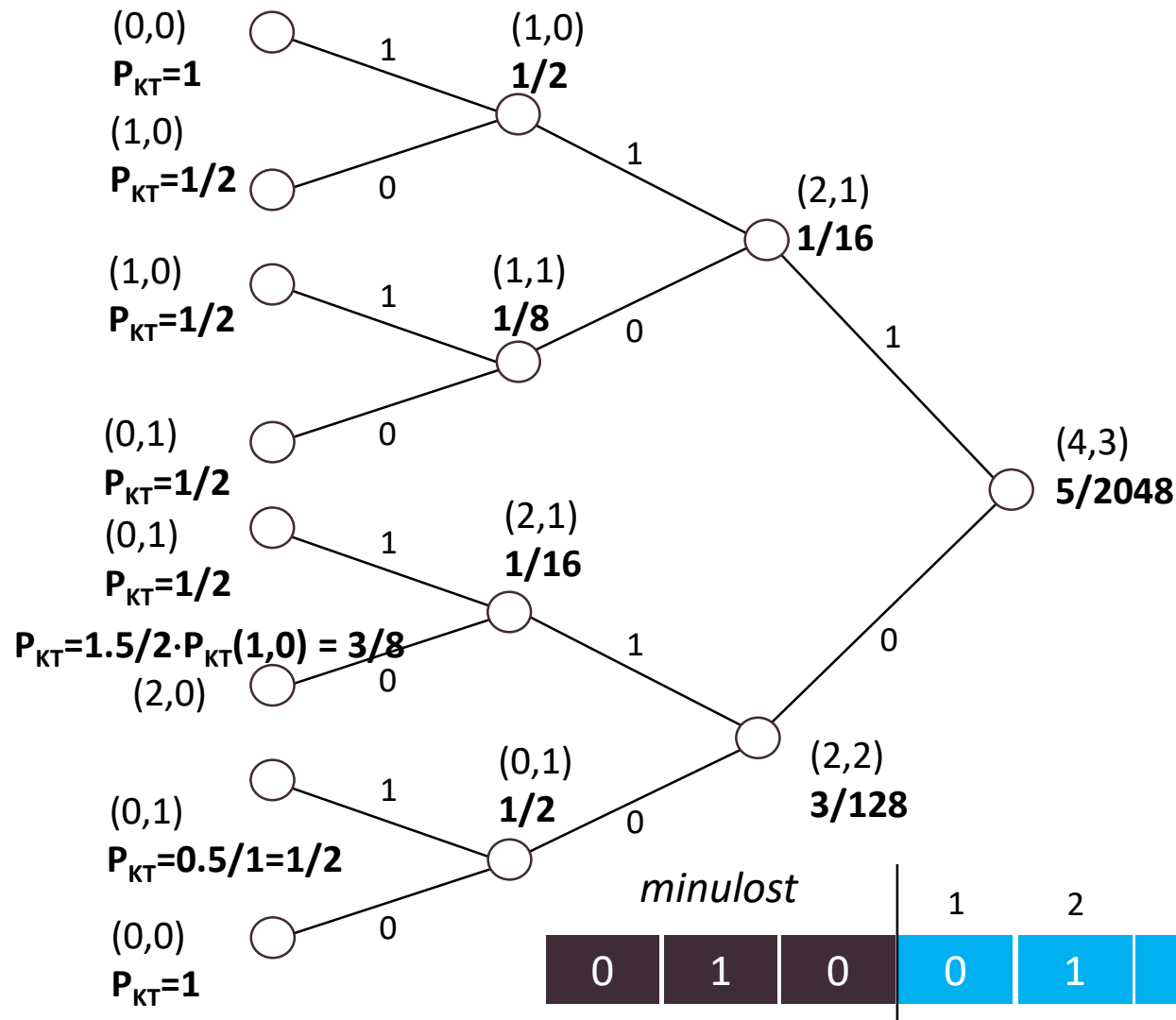
Princip určení pravděpodobnosti pomocí CTW

- Uvažujme váhovaný CT T_3 (tj. $D = 3$) pro sekvenci $h_i^T = 0110100$ a $x_{i-D}^0 = \dots 010$.
- 2) Přiřadíme počítadla $(a,b) = (\text{počet nul, počet jedniček})$



Princip určení pravděpodobnosti pomocí CTW

- Uvažujme váhovaný CT T_3 (tj. $D = 3$) pro sekvenci $h_i^T = 0110100$ a $x_{i-D}^0 = \dots 010$.
- 3) Určíme P_{KT} a P_s (postupujeme od listů směrem ke kořenu)



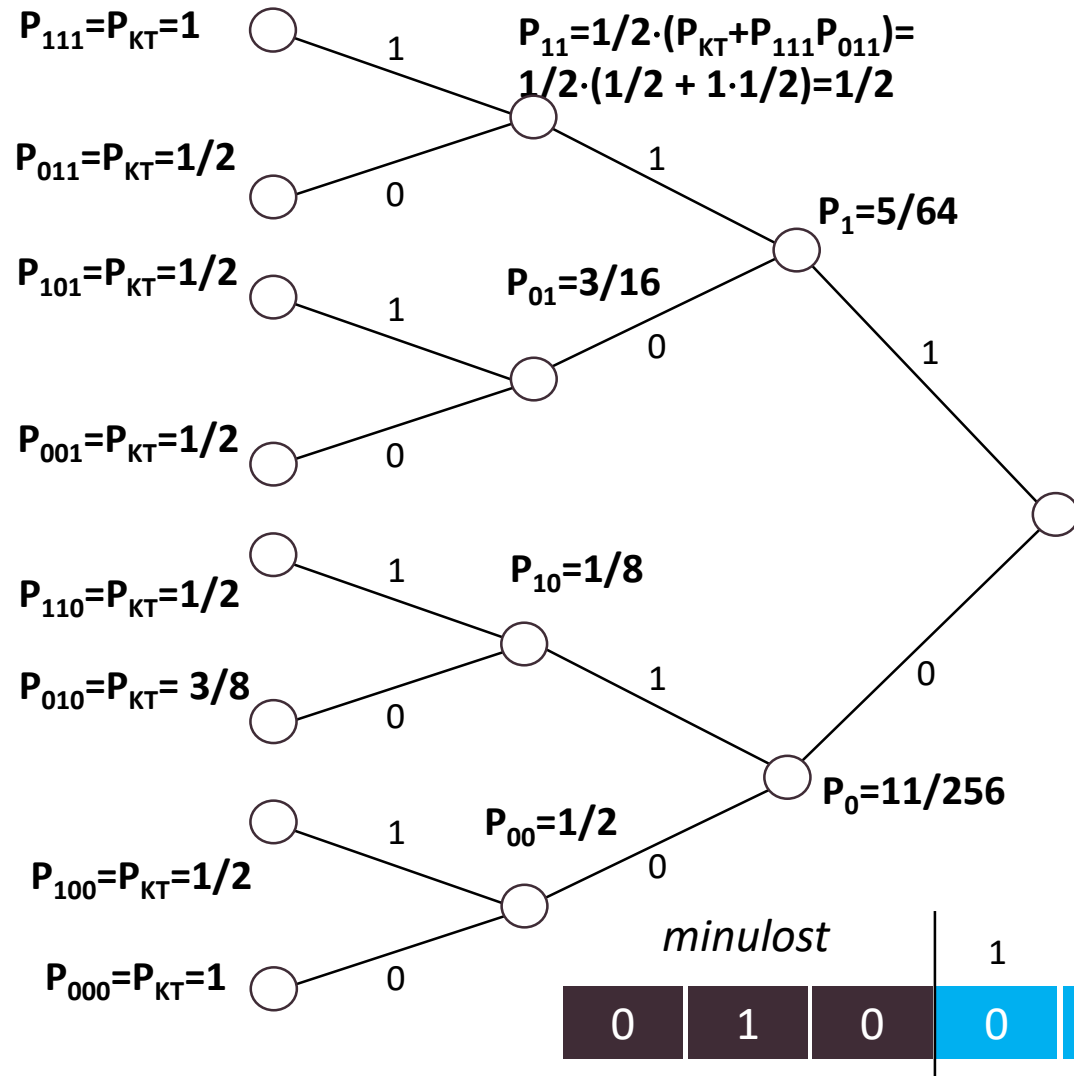
$$P_{kt}(0,0) = 1$$

$$P_{kt}(a+1, b) = \frac{a + \frac{1}{2}}{a + b + 1} \cdot P_{kt}(a, b)$$

$$P_{kt}(a, b+1) = \frac{b + \frac{1}{2}}{a + b + 1} \cdot P_{kt}(a, b)$$

Princip určení pravděpodobnosti pomocí CTW

- Uvažujme váhovaný CT T_3 (tj. $D = 3$) pro sekvenci $h_i^T = 0110100$ a $x_{i-D}^0 = \dots 010$.
- 3) Určíme P_{KT} a P_s (postupujeme od listů směrem ke kořenu)



Listový uzel:

$$P_s = P_{kt}(a_s, b_s)$$

Ostatní uzly:

$$P_s = \frac{1}{2} (P_{kt}(a_s, b_s) + P_{0s}P_{1s})$$

$$P_{CTW} = 1/2 \cdot (5/2048 + 5/64 \cdot 11/256) = 95/32768$$

P_{CTW} je pravděpodobnost že následující bit bude 1

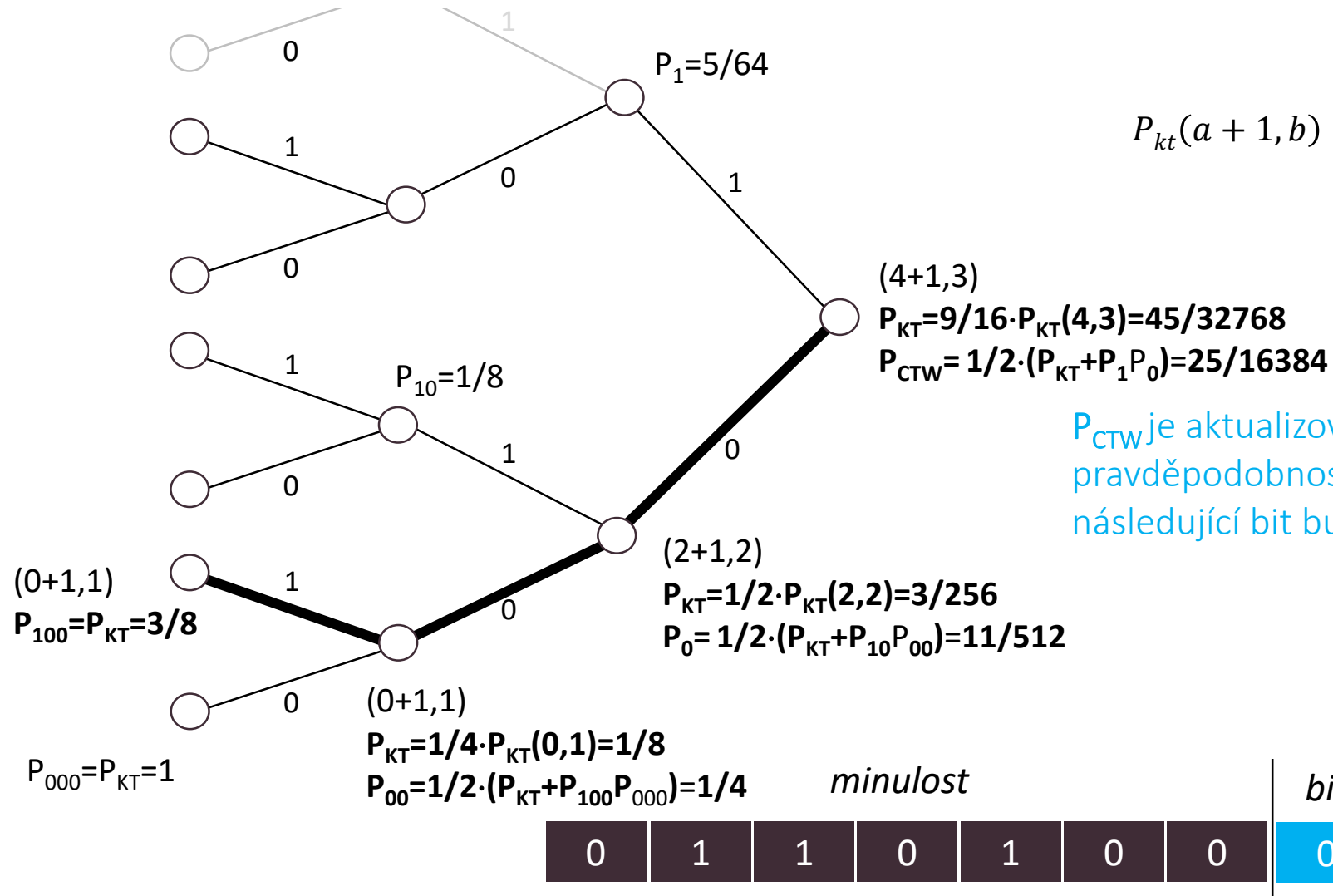
$$\text{Prob}(1 | \dots 0100110100) = 95/32768$$

$$-\log_2 0.0028991 = 8.43$$

minulost			1	2	3	4	5	6	7	8
0	1	0	0	1	1	0	1	0	0	=1?

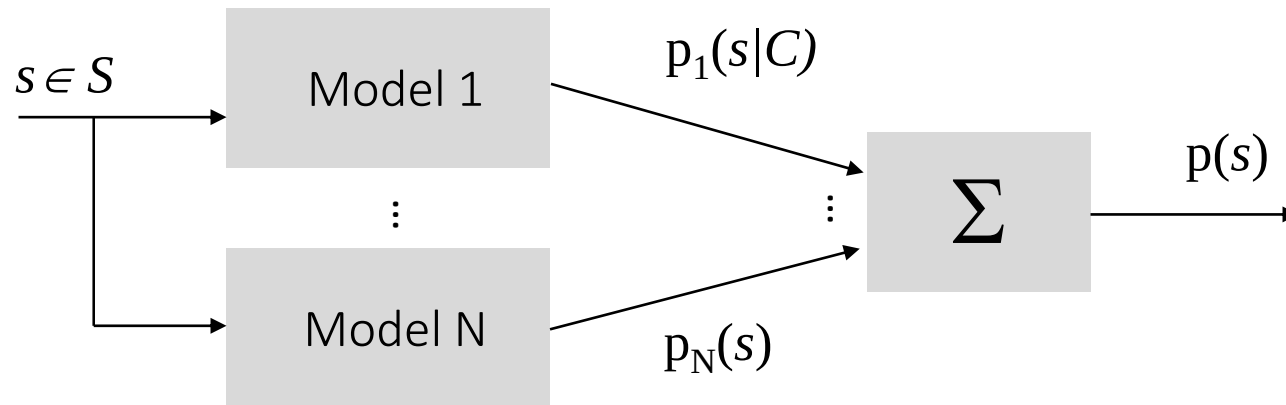
Princip aktualizace CTW

- Uvažujme váhovaný CT T_3 (tj. $D = 3$) pro sekvenci $h_i^T = 0110100$ a $x_{i-D}^0 = \dots 010$.
- Uvažujme, že následující bit bude 0. Strom je aktualizován následovně



Mixování kontextů (Context mixing)

- Mixování kontextů je kompresní metoda, kde **predikce** dalšího symbolu je **získána kombinací** dvou nebo více nezávislých **modelů**.



- Kombinování kontextů je použito v sérii algoritmů PAQ, které podobně jako CTW nebo DMC pracují po bitech. **Váhy** však nejsou **přiřazeny** kontextům, ale jednotlivým **modelům**. Navíc kontexty se nemusí mixovat směrem od nejdelšího po nejkratší řád kontextu.

Mixování kontextů

- Algoritmy mixování kontextů (ZPAQ apod.) dosahují špičkové výsledky co se týče kompresního poměru, ale jsou velmi pomalé.

Plugin	Codec	Level	Compression Ratio ^	Compression Speed	Decompression Speed
zpaq	zpaq	5	4.07	122.74 KiB/s	123.6 KiB/s
zpaq	zpaq	4	3.91	450.65 KiB/s	471.98 KiB/s
bsc	bsc		3.75	4.84 MiB/s	8 MiB/s
bzip2	bzip2	5	3.52	5.37 MiB/s	13.71 MiB/s
bzip2	bzip2	9	3.52	5.31 MiB/s	14.62 MiB/s
bzip2	bzip2	8	3.52	5.33 MiB/s	13.25 MiB/s
bzip2	bzip2	7	3.52	5.38 MiB/s	14.29 MiB/s
bzip2	bzip2	4	3.52	5.41 MiB/s	14.74 MiB/s
bzip2	bzip2	3	3.52	5.37 MiB/s	13.92 MiB/s
bzip2	bzip2	6	3.52	5.38 MiB/s	14.09 MiB/s
bzip2	bzip2	2	3.52	5.37 MiB/s	13.8 MiB/s
bzip2	bzip2	1	3.31	5.3 MiB/s	15.04 MiB/s
brotli	brotli	11	3.23	212.83 KiB/s	101.36 MiB/s
brotli	brotli	10	3.23	212.3 KiB/s	102.79 MiB/s
lzma	lzma1	6	3.14	1.26 MiB/s	26.09 MiB/s
lzma	lzma1	7	3.14	1.04 MiB/s	26.15 MiB/s

<https://quixdb.github.io/squash-benchmark>



Mixování kontextů

- DMC, PPM a CTW jsou založeny na předpokladu, že nejlepším prediktorem je nejdelší kontext, pro který je k dispozici statistika.
- Pro text je to obvykle pravda, ale
 - v audio souboru bude lepší používat prediktor, který bude u kontextů jednotlivých vzorků zanedbávat nejnižší bity, protože je to většinou šum.
 - v případě zpracování obrazů bude lepší použít prediktory zohledňující sousední pixely ve dvou rozměrech, což netvoří souvislý kontext.
 - kompresi textu lze vylepšit použitím kontextů, které leží na hranicích slov a ztotožňují velká a malá písmena.
 - v datech s pevnou délkou záznamu jako jsou databáze je užitečným kontextem číslo sloupce, někdy v kombinaci se sousedními daty ve dvou rozměrech.
- Kompresory založené na PAQ mohou mít desítky až stovky různých modelů k predikci příštího vstupního bitu. Mixování kontextů je založeno na skutečnosti, že kombinovaná predikce nezávislých modelů komprimuje lépe, než kterákoliv ze součástí použitá samostatně.

Mixování kontextů

- Předpokládejme dva prediktory pravděpodobnosti, že příští bit y bude 1 pro dané kontexty A a B : $p_a = P(y=1 | A)$ a $p_b = P(y=1 | B)$ Jaká je $p = P(y=1 | A, B)$?
- Předpokládejme, že A a B nastaly dostatečně často, aby tyto dva modely poskytovaly spolehlivé odhady, avšak oba kontexty se dosud nevyskytly společně. Bohužel neexistuje jednoznačný předpis, jak modely kombinovat.
- Základním přístupem je využít **průměrování**, tj. $p = 1/2 (p_a + p_b)$ To však předpokládá, že výsledný efekt je součtem.
- Když $p_a=0,5$ a $p_b=0,99$, pak se intuitivně zdá, že p_b udává vyšší důvěru pro predikci, takže musí dostat větší váhu. V tomto případě by bylo výhodnější využít **váhové průměrování**
$$p = \alpha p_a + (1 - \alpha) p_b$$
 - Váhování lze založit na tom, kolikrát jsme danou situaci v minulosti zaznamenali. Čím vyšší počítadlo, tím vyšší váhu by měl prediktor mít. Může však nastat problém při změně charakteru dat (nově příchozí by měly mít vyšší váhu).

Lineární mixování

■ Starší verze PAQ používají váhované **lineární mixování**

- PAQ6 používá k modelů určujících pravděpodobnost na základě počtu jedniček $n_1^{(i)}$ a nul $n_0^{(i)}$ takto:
 $P^{(i)}(1) = n_1^{(i)} / (n_0^{(i)} + n_1^{(i)})$, kde $i = 1 \dots k$
- Pravděpodobnosti se kombinují váhovým součtem počtů. Váhy jsou nastaveny tak, aby se minimalizovala cena za kódování v prostoru vah.

$$P(1) = \frac{S_1}{S_1 + S_0} \qquad S_0 = \varepsilon + \sum_i w_i n_0^{(i)} \qquad S_1 = \varepsilon + \sum_i w_i n_1^{(i)}$$

w_i je nezáporná váha i -tého modelu a ε malá kladná konstanta, potřebná k zabránění degeneraci chování, když je S blízko 0

- Aktualizace optimální váhy se dá najít z **částečné ceny za kódování vzhledem k w_i** . Cena za kódování 0 je $-\log p_1$, za 1 je to $-\log p_0$. Výsledkem je, že po zakódování bitu y (0 nebo 1) se váhy aktualizují postupem podél **cenového gradientu v prostoru vah**:

$$w_i = \max \left[0, w_i + E[(S_0 + S_1)n_1^{(i)} - S_1(n_0^{(i)} + n_1^{(i)})] / (S_0 S_1) \right], \quad E = y - P(1)$$

kde E je chyba predikce

- Počty se upravují ve prospěch nových dat. Dvojice (n_0, n_1) reprezentuje stav (podobně jako u dvoustupňového modelu).

Mixování neuronovými sítěmi

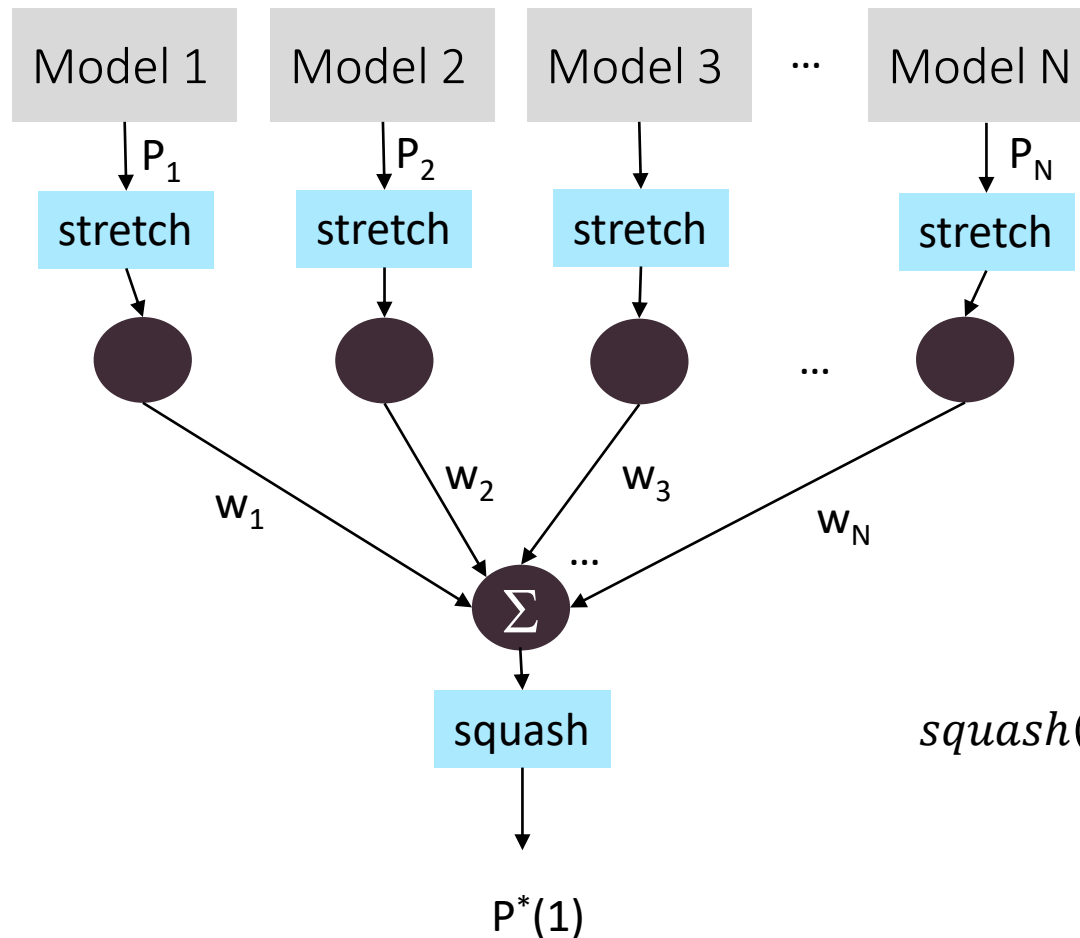
- Vstupem mixeru je množina predikcí z jednotlivých modelů, výstupem mixeru je opravená predikce.
 - Předpokladem je, že seskupení modelů bude dávat vyšší přesnost nežli jednotlivé modely. Čím větší je chyba predikce E , tím více je nutné během aktualizace vah konkrétní váhu korigovat. Analogický princip je použit i v neuronových sítích a učícím algoritmu Back Propagation.

$$w_m = w_m + error_m \cdot \alpha \cdot Input_i$$

- Pro mixování je v PAQ použita jednoduchá neuronová síť bez skrytých vrstev.
 - Vstupem jsou výstupy jednotlivých modelů, tj. množina predikcí pravděpodobností P_i , že další bit bude 1.
 - Na vstupy je aplikována **logistická funkce** $stretch(p)=\log(p/(1-p))$.
 - Získané hodnoty jsou váhovány koeficienty w_i , sečteny a na výstup je aplikována reverzní transformace **squash**(x)= $1/(1 + e^{-x})$, tj. $p = squash(\sum w_i stretch(P_i))$
 - Hledání optimální aktualizace vah se provádí s využitím částečných derivací ceny za kódování s ohledem na váhy $w_i = w_i + \lambda E stretch(P_i)$, kde λ je rychlost učení, typicky kolem 0,01 a $E=(y - p)$ je predikční chyba.
 - Na rozdíl lineárního mixování mohou být váhy záporné.

Mixování neuronovými sítěmi

- V PAQ / ZPAQ jsou funkce `squash()` a `stretch()` implementovány pomocí lookup tabulek, kde jsou uložena 12-bit / 15-bit čísla s pevnou čárkou.

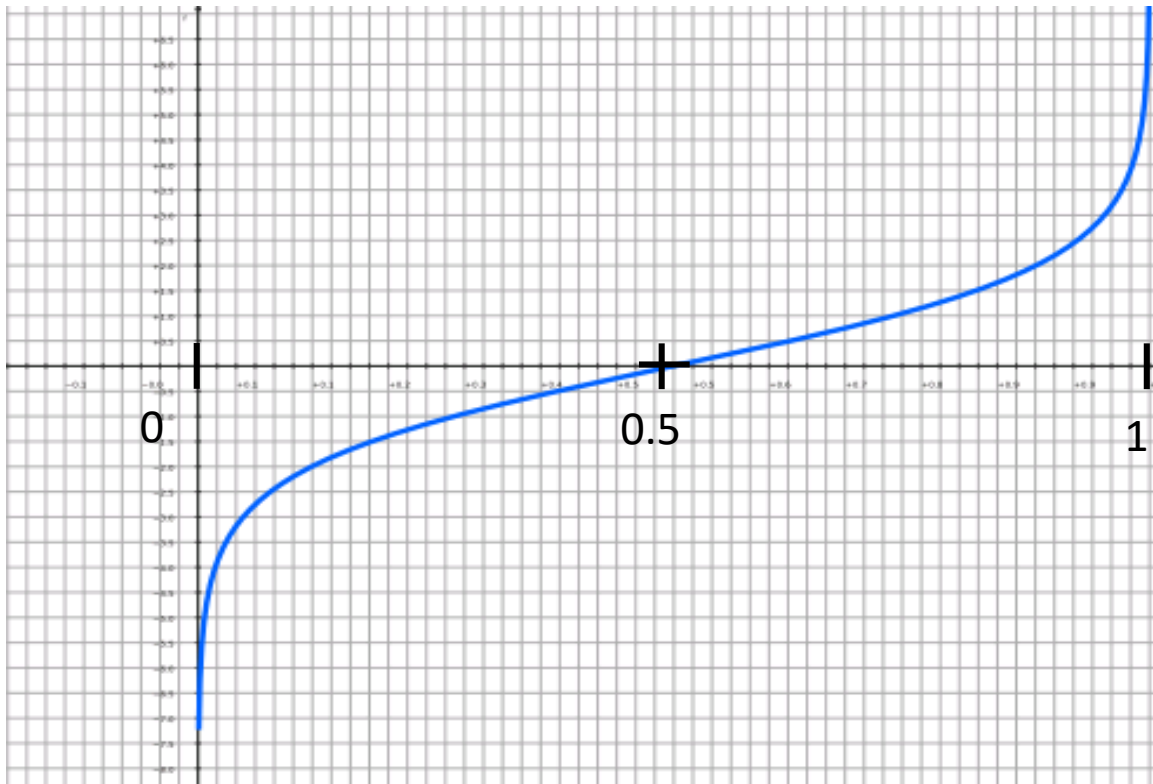


$$\text{stretch}(x) = \ln \frac{x}{1-x}$$

$$\text{squash}(x) = \text{stretch}^{-1}(x) = \frac{1}{1 + e^{-x}}$$

Mixování neuronovými sítěmi

- Převod do logistické domény má následující vlastnost: model jehož výstup P_i je hodnota blízká 0 nebo hodnota blízká 1 bude mít přiřazen vyšší význam. Naopak, logistická funkce přiřazuje nejmenší (nulový) význam těm modelům, které dávají $p_i=0.5$.



PAQ

- PAQ (od 2002) je série archivačních kompresních programů, které dávají nejlepší kompresní poměr. Tyto programy byly vyvinuty a odladěny na sadě benchmarkových úloh.
- Jedná se o Open source software, dostupný pod GNU General Public License a dostupný na <http://mattmahoney.net/dc/paq.html>.
- PAQ používá **mixování kontextu** a sbírá *statistiky příštích symbolů* pro následující kontexty:
 - n -gramy. Kontext je posledních n bytů před predikovaným znakem (jako v PPM).
 - n -gramy celých slov, kdy se ignoruje, zda jsou písmena malá nebo velká (užitečné v textových souborech).
 - „řídke“ kontexty, například druhý a čtvrtý byte, předcházející predikovaný znak (používá se u některých binárních souborů).
 - „analogové“ (souvislé) kontexty, tvořené vyššími bity předcházejících 8- nebo 16-bitových slov (užitečné pro multimediální soubory).
 - dvourozměrné kontexty (užitečné pro obrázky, tabulky a tabulkové procesory). Délka řádku se určí nalezením délky kroku opakujících se bytových vzorků.
 - další specializované modely pro exe-soubory x86, nebo obrázky v BMP, TIFF nebo JPEG. Tyto modely jsou aktivní pouze v případě detekování určitého typu souboru.

PAQ – kombinování predikcí

- Všechny verze PAQ predikují a komprimují v každém okamžiku jeden bit, ale liší se detailech modelů a jak se predikce kombinují a na závěr zpracují. Pro kódování se používá aritmetický kodér.
- Podle verze jsou tři metody *kombinování predikcí*
 - V PAQ1 až PAQ3 se každá predikce reprezentuje jako dvojice bitových počtů (n_0, n_1). Tyto počty se kombinují váhovým součtem s většími váhami přidělenými delším kontextům.
 - V PAQ4 až PAQ6 se predikce kombinují stejně jako u předchozí, ale váhy přidělené každému modelu se nastavují ve prospěch přesnějšího modelu PAQ_i.
 - PAQ7 a výše model nevytváří dvojici, ale jednu pravděpodobnost. Pravděpodobnosti se kombinují pomocí umělé neuronové sítě.
- PAQ1SSE a další verze zpracovávají na závěr predikci pomocí druhé estimace symbolu SSE. Kombinovaná predikce a krátký kontext se používá k nalezení nové predikce v tabulce. Po zakódování bitu se položka tabulky nastaví tak, aby se snížila predikční chyba. Stupně SSE se mohou řetězit s různými kontexty nebo počítat paralelně s průměrováním výstupů.

PAQ – preprocessing

- Pokročilejší verze PAQ, zejména PAsQ, PAQAR (obě jsou odvozené z PAQ6) a PAQ8HP1 až PAQ8HP8 (odvozené z PAQ8 - odměněné Hutterovou cenou 50k€ for enwik 100) předzpracovávají textové soubory.
- Vyhledávají se slova v externím slovníku a nahrazují se 1 – 3 bytovými kódy. V sérii PAQ8HP je slovník organizován vytvářením skupin ze syntakticky a sémanticky podobných slov. To umožňuje modelům používat jako kontext pouze nejvyšší významové bity kódů ze slovníku.
- Velká písmena se zakódují speciálním znakem následovaným malým písmenem.